

Guru Nanak Institute of Technology



Computer Programming Notes (JNTUH)

Ist Year B. Tech (CSE / IT)

FACULTY: - J.NARESH KUMAR
Assistant Professor, IT Dept.



COMPUTER PROGRAMMING

Objectives:

- To understand the various steps in Program development.
- To understand the basic concepts in C Programming Language.
- To learn how to write modular and readable C Programs
- To learn to write programs (using structured programming approach) in C to solve problems.
- To introduce the students to basic data structures such as lists, stacks and queues.
- To make the student understand simple sorting and searching methods.

UNIT - I

Introduction to Computers – Computer Systems, Computing Environments, Computer Languages, Creating and running programs, Program Development. Algorithm and flowcharts, Number systems-Binary, Decimal, Hexadecimal and Conversions, storing integers and real numbers.

Introduction to C Language – Background, C Programs, Identifiers, Types, Variables, Constants, Input / Output, Operators(Arithmetic, relational, logical, bitwise etc.), Expressions, Precedence and Associativity, Expression Evaluation, Type conversions, Statements- Selection Statements(making decisions) – if and switch statements, Repetition statements (loops)-while, for, do-while statements, Loop examples, other statements related to looping – break, continue, goto, Simple C Program examples.

UNIT - II

Functions-Designing Structured Programs, Functions, user defined functions, inter function communication, Standard functions, Scope, Storage classes-auto, register, static, extern, scope rules, type qualifiers, recursion-recursive functions, Limitations of recursion, example C programs.

Arrays – Concepts, using arrays in C, inter function communication, array applications-linear search, binary search and bubble sort,, two – dimensional arrays, multidimensional arrays, C program examples.

UNIT - III

Pointers – Introduction (Basic Concepts), Pointers for inter function communication, pointers to pointers, compatibility, Pointer Applications-Arrays and Pointers, Pointer Arithmetic and arrays, Passing an array to a function, memory allocation functions, array of pointers, programming applications, pointers to void, pointers to functions.

Strings – Concepts, C Strings, String Input / Output functions, arrays of strings, string manipulation functions, string /data conversion, C program examples.

UNIT - IV

Enumerated, Structure ,and Union Types– The Type Definition(typedef), Enumerated types, Structures – Declaration, initialization, accessing structures, operations on structures, Complex structures-Nested structures, structures containing arrays, structures containing pointers, arrays of structures, structures and functions, Passing structures through pointers, self referential structures, unions, bit fields, C programming examples, command –line arguments, Pre-processor commands.

UNIT – V

Input and Output – Concept of a file, streams, text files and binary files, Differences between text and binary files, State of a file, Opening and Closing files, file input / output functions (standard library input / output functions for files), file status functions (error handling),Positioning functions(fseek, rewind and ftell), C program examples.

TEXT BOOKS:

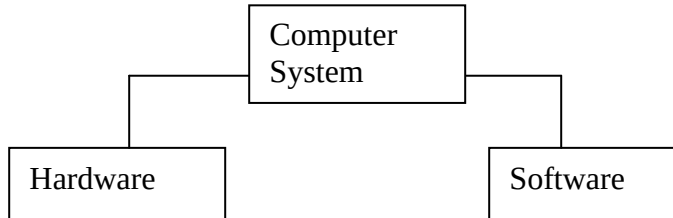
1. Computer Science: A Structured Programming Approach Using C, B.A.Forouzan and R.F. Gilberg, Third Edition, Cengage Learning.
2. Programming in C. P. Dey and M Ghosh , Oxford University Press.

REFERENCE BOOKS:

1. The C Programming Language, B.W. Kernighan and Dennis M.Ritchie, PHI.
2. C& Data structures – P. Padmanabham, Third Edition, B.S. Publications

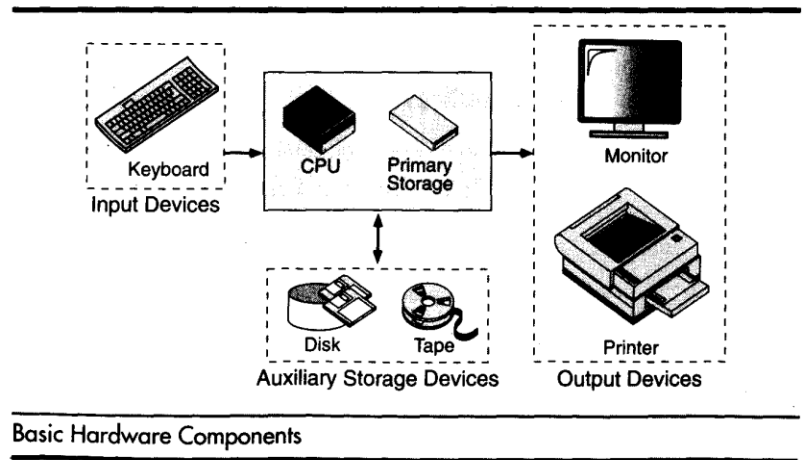
UNIT – IIntroduction to Computers**Computer Systems:**

A computer is a system made of two major components: hardware and software. The computer hardware is the physical equipment. The software is the collection of programs (instructions) that allow the hardware to do its job.

**Computer Hardware**

The hardware component of the computer system consists of five parts: input devices, central processing unit (CPU), primary storage, output devices, and auxiliary storage devices.

The **input device** is usually a keyboard where programs and data are entered into the computers. Examples of other input devices include a mouse, a pen or stylus, a touch screen, or an audio input unit.



The **central processing unit (CPU)** is responsible for executing instructions such as arithmetic calculations, comparisons among data, and movement of data inside the system. Today's computers may have one, two, or more CPUs. **Primary storage**, also known as main memory, is a place where the programs and data are stored temporarily during processing. The data in primary storage are erased when we turn off a personal computer or when we log off from a time-sharing system.

The **output device** is usually a monitor or a printer to show output. If the output is shown on the monitor, we say we have a **soft copy**. If it is printed on the printer, we say we have a hard copy.

Auxiliary storage, also known as **secondary storage**, is used for both input and output. It is the place where the programs and data are stored permanently. When we turn off the computer, or programs and data remain in the secondary storage, ready for the next time we need them.

Computer Software

Computer software is divided into two broad categories: system software and application software. System software manages the computer resources. It provides the interface between the hardware and the users. Application software, on the other hand is directly responsible for helping users solve their problems.

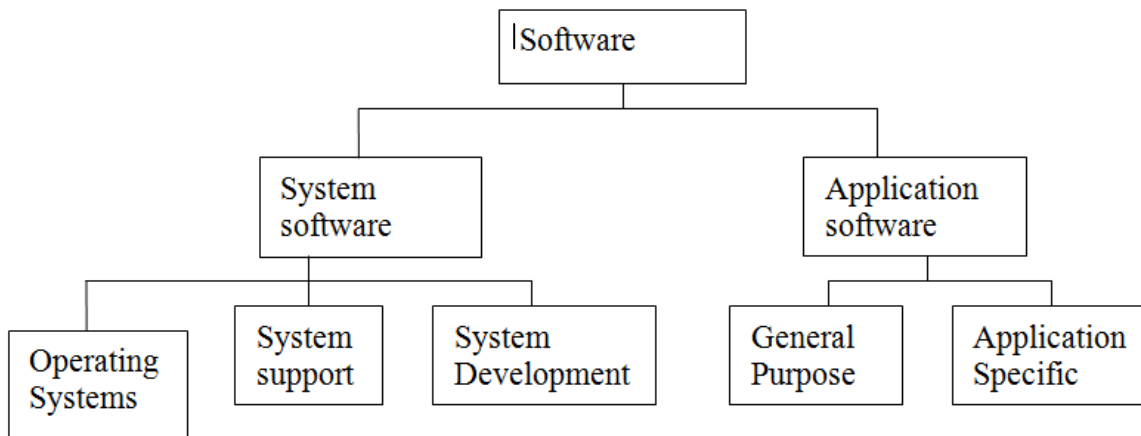


Fig: Types of software

System Software:

System software consists of programs that manage the hardware resources of a computer and perform required information processing tasks. These programs are divided into three classes: the operating system, system support, and system development.

The **operating system** provides services such as a user interface, file and database access, and interfaces to communication systems such as Internet protocols. The primary purpose of this software is to keep the system operating in an efficient manner while allowing the users access to the system.

System support software provides system utilities and other operating services. Examples of system utilities are sort programs and disk format programs. Operating services consists of programs that provide performance statistics for the operational staff and security monitors to protect the system and data.

The last system software category, **system development software**, includes the language translators that convert programs into machine language for execution ,debugging tools to ensure that the programs are error free and computer –assisted software engineering(CASE) systems.

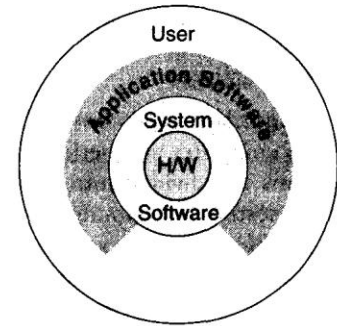
Application software

Application software is broken in to two classes :general-purpose software and application –specific software. **General purpose software** is purchased from a software developer and can be used for more than one application. Examples of general purpose software include word processors, database management systems and computer aided design systems. They are labeled general purpose because they can solve a variety of user computing problems.

Application –specific software can be used only for its intended purpose.A general ledger system used by accountants and a material requirements planning system used by a manufacturing organization are examples of application-specific software. They can be used only for the task for which they were designed they cannot be used for other generalized tasks.

The relationship between system and application software is shown in fig-2. In this figure, each circle represents an interface point. The inner core is hardware. The user is represented by the outer layer. To work with the system, the typical user uses some form of application software.

The application software in turn interacts with the operating system, which is a part of the system software layer. The system software provides the direct interaction with the hardware. The opening at the bottom of the figure is the path followed by the user who interacts directly with the operating system when necessary.



Relationship Between System and Application Software

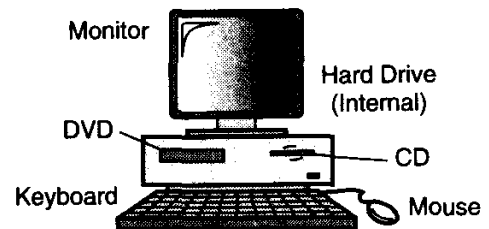
Computing Environments:

In the early days of computers, there was only one environment: the main frame computer hidden in a central computing department. With the advent of mini computers and personal computers, the environment changed, resulting in computers on virtually every desktop.

Personal Computing Environment

In 1971, Marcian E. Hoff, working for Intel, combined the basic elements of the central processing unit into the microprocessor. The first computer on a chip was the Intel 4004 and was the grandparent many times removed of Intel's current system.

If we are using a personal computer, all of the computer hardware components are tied together in our personal computer (PC).

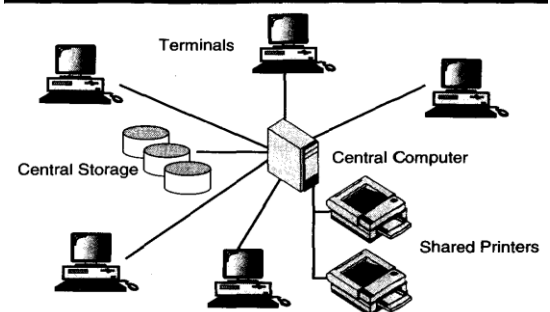


Personal Computing Environment

Time-Sharing Environment

Employees in large companies often work in what is known as a **time-sharing environment**. In the time-sharing environment, many users are connected to one or more computers. These computers may be minicomputers or central mainframes. The terminals they use are often nonprogrammable, although today we see more and more microcomputers being used to simulate terminals. Also, in the time-sharing environment, the output devices and auxiliary storage devices are shared by all of the users. A typical college lab in which a minicomputer is shared is shared by many students is shown in figure :

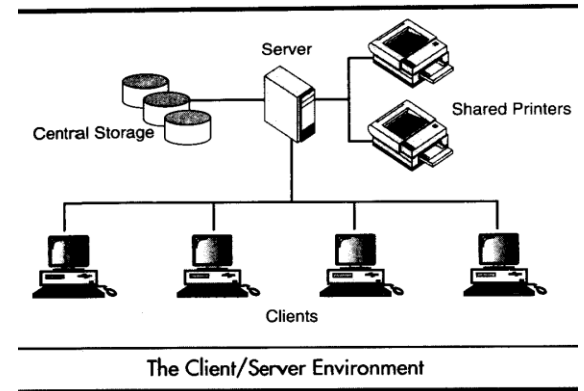
In the time-sharing environment, all computing must be done by the central computer. The central computer has many duties: It must control the shared resources; it must manage the shared data and printing and it must do the computing.



Time-sharing Environment

Client/Server Environment

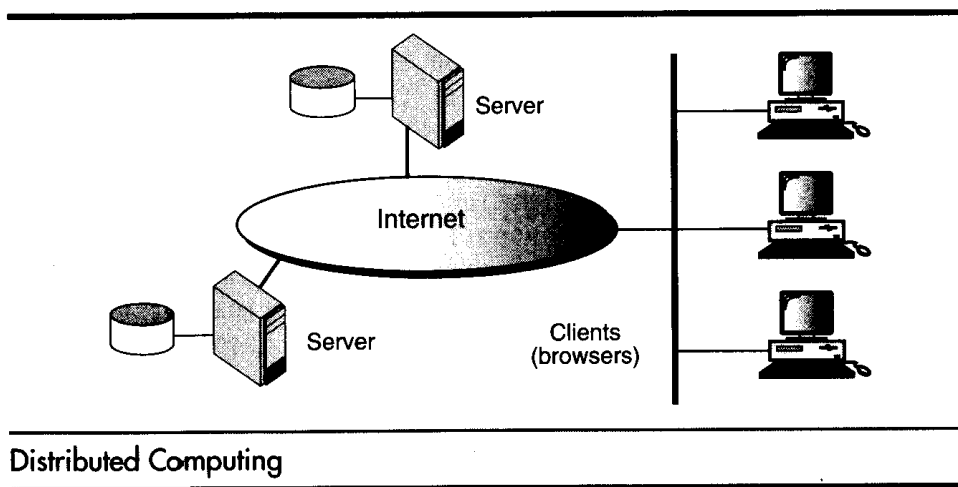
A **client/server** computing environment splits the computing function between a central computer and users' computers. The users are given personal computers or work stations so that some of the computation responsibility can be moved from the central computer and assigned to the workstations. In the client-server environment, the users' micro computers or workstations are called the **client**. The central computer, which may be a powerful microcomputer, minicomputer, or central mainframe system, is known as the **server**. Because the work is now shared between the users' computers and the central computer, response time and monitor display are faster and the users are more productive.



Distributed Computing

A **Distributed Computing** environment provides a seamless integration of computing functions between different servers and clients. The internet provides connectivity to different servers throughout the world. For example eBay uses several computers to provide its auction services. This environment provides a reliable, scalable, and highly available network.

Fig: Distributed Computing



Computer Languages:

To write a program for a computer, we must use a **computer language**. Over the years computer languages have evolved from machine languages to natural languages.

1940's	Machine level Languages
1950's	Symbolic Languages
1960's	High-Level Languages

Machine Languages

In the earliest days of computers, the only programming languages available were machine languages. Each computer has its own machine language, which is made of streams of 0's and 1's.

Instructions in machine language must be in streams of 0's and 1's because the internal circuits of a computer are made of switches transistors and other electronic devices that can be in one of two states: off or on. The off state is represented by 0, the on state is represented by 1.

The only language understood by computer hardware is machine language.

Symbolic Languages:

In early 1950's Admiral Grace Hopper, A mathematician and naval officer developed the concept of a special computer program that would convert programs into machine language. The early programming languages simply mirror to the machine languages using symbols of mnemonics to represent the various machine language instructions because they used symbols, these languages were known as symbolic languages.

Computer does not understand symbolic language it must be translated to the machine language. A special program called assembler translates symbolic code into machine language. Because symbolic languages had to be assembled into machine language they soon became known as assembly languages.

Symbolic language uses symbols or mnemonics to represent the various, machine language instructions.

High Level Languages:

Symbolic languages greatly improved programming efficiency; they still required programmers to concentrate on the hardware that they were using. Working with symbolic languages was also very tedious because each machine instruction has to be individually coded. The desire to improve programmer efficiency and to change the focus from the computer to the problem being solved led to the development of high-level language.

High level languages are portable to many different computers, allowing the programmer to concentrate on the application problem at hand rather than the intricacies of the computer. High-level languages are designed to relieve the programmer from the details of the assembly language. High level languages share one thing with symbolic languages, They must be converted into machine language. The process of converting them is known as compilation.

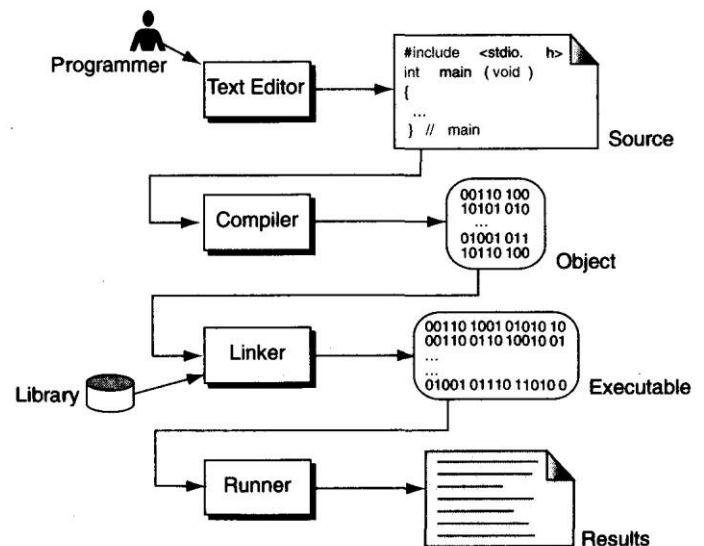
The first widely used high-level languages, FORTRAN (FORMula TRANslation) was created by John Backus and an IBM team in 1957; it is still widely used today in scientific and engineering applications. After FORTRAN was COBOL (Common Business-Oriented Language). Admiral Hopper was played a key role in the development of the COBOL Business language.

C is a high-level language used for system software and new application code.

Creating and Running Programs:

Computer hardware understands a program only if it is coded in its machine language. It is the job of the programmer to write and test the program. There are four steps in this process:

1. Writing and Editing the program
2. Compiling the program
3. Linking the program with the required library modules
4. Executing the program.



1. Writing and Editing Program

The software used to write programs is known as a **text editor**. A text editor helps us enter, change, and store character data. Depending on the editor on our system, we could use it to write letters, create reports, or write programs. The main difference between text processing and program writing is that programs are written using lines of code, while most text processing is done with character and lines.

Text editor is a generalized word processor, but it is more often a special editor included with the compiler. Some of the features of the editor are search commands to locate and replace statements, copy and paste commands to copy or move statements from one part of a program to another, and formatting commands that allow us to set tabs to align statements.

After completing a program, we save our file to disk. This file will be input to the compiler; it is known as a **source file**.

2. Compiling Programs:

The code in a source file stored on the disk must be translated into machine language, This is the job of the **compiler**. The c compiler is two separate programs. the **preprocessor** and the **translator**.

The preprocessor reads the source code and prepares it for the translator. While preparing the code ,it scans for special instructions known as preprocessor commands. These commands tell the preprocessor to look for special code libraries, make substitutions in the code and in other ways prepare the code for translation into machine language. The result of preprocessing is called the translation unit.

After the preprocessor has prepared the code for compilation, the translator does the actual work of converting the program into machine language. The translator reads the translation unit and writes the resulting object module to a file that can then be combined with other precompiled units to form the final program. An object module is the code in machine language. The output of the compiler is machine language code, but it is not ready to run; that is ,it is not executable because it does not have the required C and other functions included.

3. Linking Programs:

A C program is made up of many functions. We write some of these functions, and they are a part of our source program. There are other functions, such as input/output processes and, mathematical library functions, that exist elsewhere and must be attached to our program. The linker assembles all of these functions, ours and systems into our final executable program.

4. Executing Programs:

Once program has been linked, it is ready for execution. To execute a program we use an operating system command, such as run, to load the program into primary memory and execute it. Getting the program into memory is the function of an operating system program known as the loader. It locates the executable program and reads it into memory. When everything is loaded, the program takes control and it begins execution.

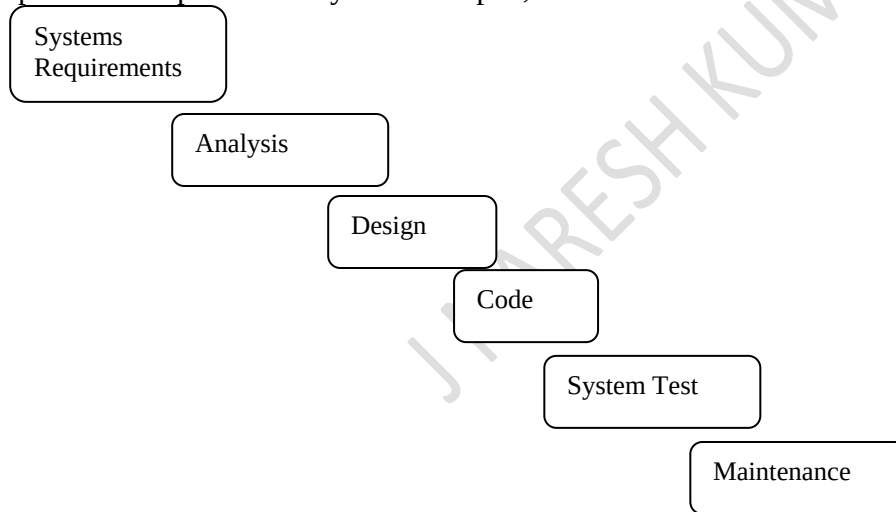
In a typical program execution, the reads data for processing ,either from the user or from a file. After the program processes the data, it prepares the output. at output can be to the user's monitor or to a file. When the program has finished its job, it tells the operating system ,which then removes the program from memory.

System Development Method

A **software development process** is a structure imposed on the development of a software product. This critical process determines the overall quality and success of our program. If we carefully design each program using good structured development techniques, programs will be efficient, error-free, and easy to maintain.

System Development Life Cycle

Today's large-scale modern programming projects are built using a series of interrelates phases commonly referred to as the system development cycle. Although the exact number and names of the phases differ depending on the environment there is general agreement as to the steps that must be followed. One very popular development life cycle developed, this modal consists of between 5 and 7 phases.



The water fall modal starts with systems requirements in this phase the systems analyst defines requirements that specify what the proposed system is to accomplish. The requirements are usually stated in terms that the user understands. The analysis phase looks at different alternatives from a systems point of view while the design phase determined how the system will be built. In the design phase the functions of the individual programs that will make up the system are determined and the design of the files and / or the databases is completed. Finally in the 4th phase code, we write the programs. After the programs have been written and tested to the programmer's satisfaction, the project proceeds to the system test. All of the programs are tested together to make sure of the system works as a whole. The final phase maintenance keeps the system working once it has been put into production.

Although the implication of the water falls approach is that the phases flow in a continuous stream from the first to the last, this is not really the case. As each phase is developed, errors and omissions will often be found in the previous work. When this happens it is necessary to go back to the previous phase to rework it for consistency and to analyze the impact caused by the changes.

ALGORITHM /PSEUDOCODE:**PSEUDOCODE:**

Pseudo code is an artificial and informal language that helps programmers develop algorithms. pseudocode is similar to everyday English, it is convenient and user friendly although it is not an actual computer programming language. Psuedocode programs are not actually executed on computer rather they merely help the programmer “think out” a program before attempting to write it in a programming language such as C.

ALGORITHM:

Algorithms was developed by an Arab mathematician. It is chalked out step-by-step approach to solve a given problem. It is represented in an English like language and has some mathematical symbols like $->$, $>$, $<$, $=$ etc. To solve a given problem or to write a program you approach towards solution of the problem in a systematic, disciplined, non-adhoc, step-by-step way is called Algorithmic approach. Algorithm is a penned strategy(to write) to find a solution.

Example: Algorithm/pseudo code to add two numbers

Step 1: Start
 Step 2: Read the two numbers in to a,b
 Step 3: $c=a+b$
 Step 4: write/print c
 Step 5: Stop.

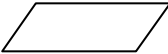
FLOW CHART :

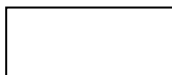
A Flow chart is a Graphical representation of an Algorithm or a portion of an Algorithm. Flow charts are drawn using certain special purpose symbols such as Rectangles, Diamonds, Ovals and small circles. These symbols are connected by arrows called flow lines.

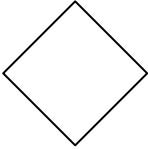
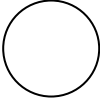
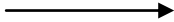
(or)

The diagrammatic representation of way to solve the given problem is called flow chart.

The following are the most common symbols used in Drawing flowcharts:

Oval		Terminal	start/stop/begin/end.
Parallelogram		Input/output	Making data available For processing(input) or recording of the process information(output).
Document		Print Out	show data output in the form of document.



Rectangle		Process	Any processing to be Done .A process changes or moves data.An assignment operation.
Diamond		Decision	Decision or switching type of operations.
Circle		Connector	Used to connect Different parts of flowchart.
Arrow		Flow	Joins two symbols and also represents flow of execution.

PROGRAM DEVELOPMENT STEPS :

Program Development is a multistep process that requires that we understand the problem, develop a solution, write the program, and then test it. When we are given the assignment to develop a program, we will be given a program requirements statement and the design of any program interfaces. We should also receive an overview of the complete project so that we will take the inputs we are given and convert them to the outputs that have been specified. This is known as program design.

Understand the Problem

The first step in solving any problem is to understand it. By reading the requirements statements carefully, we fully understand it, we review our understanding with the user and the systems analyst to know the exact purpose.

Develop the solution

Once we fully understand the problem we need to develop our solution. Three tools will help in this task. 1. Structure chart, 2.Pseudocode & 3.Flowcharts. Generally we will use structure chart and either flowchart or Pseudo code

The structure chart is used to design the whole program .Pseudo code and flowcharts are used to design the individual parts of the program.

Structure chart: A structure chart, also known as hierarchy chart, shows the functional flow through our program. The structure chart shows how we are going to break our program into logical steps each step will be a separate module. The structure chart shows the interaction between all the parts (modules) of our program.

We can use flowchart or pseudo code to complete the design of your program will depend on experience and difficulty of the program your designing.

Write the program

When we write a program, we start with the top box on the structure chart and work our way to the bottom. This is known as top-down implementation. We will write the programs by using structure chart and flowchart or pseudo code.

Test the Program

Program testing can be a very tedious and time-consuming part of program development. As the programmer we are responsible for completely testing our program. In large-development projects test engineers are responsible for testing to make sure all the programs work together. There are 2 types of testing.

1. **Black box testing:** This is done by the system test engineer and the user. Black box testing is the programs are tested without knowing what is inside it, without knowing how it works. Black box test plans are developed by looking only at the requirements statement. The test engineer uses these requirements to develop test plans.
2. **White box testing:** This is the responsibility of the programmer. White box testing assumes that the tester knows everything about the program.

Except for the simplest program, one set of test data will not completely validate a program.

NUMBER SYSTEMS USED IN PROGRAMMING :

The number systems give us different ways to represent numbers. For example, we use the decimal system to represent numbers, using the digits from 0 to 9. The systems that are interesting for us, in programming, are the decimal(dec), binary(bin) and hexadecimal(hex).

Base

A base of a number system (or notation) is the number of symbols that we use to represent the numbers. These symbols (digits) are often called alphabet of that system. The notations are usually named after their base – binary, octal, decimal, hexadecimal. For example the decimal numbering system uses 10 digits.

The base serves not only for naming. More importantly, it plays a role for the representation of the numbers in the positional notations.

When a number is written, its base should be written after the number in subscript. 123_{10} , 110010_2 , $1F4_{16}$. If the base is not provided, the default is that the number is in decimal.

In programming, we are interested in positional systems. In particular the most useful ones have a base that is an exact power of 2 – binary(base 2) and hexadecimal(base 16 = 2 to the power of 4).

Value representation

All positional systems that we use, represent the numbers in the same manner. To represent a given number we need to break it down to a sum of exact powers of the base of that notation. This sounds more complicated than it is. You already know how this works, because you work with numbers all the time. Here is an example:

The number $1234 = 1000 + 200 + 30 + 4$, where

$1000 = 1 * 10^3$ or in words 1 time 10 to the power of 3

$200 = 2 * 10^2$ or .. 2 times ten to the power of 2

$30 = 3 * 10^1$ or .. 3 times 10 to the power of 1

$4 = 4 * 10^0$ or .. 4 times 10 to the power of 0. Remember, each number to the power of 0 is equal to 1. So $4 = 4 * 1 = 4 * 100$

$1234 = 1 * 10^3 + 2 * 10^2 + 3 * 10^1 + 4 * 10^0$

We work with powers of 10, because 10 is the base of the system that we use. The same rule applies to any other positional number system. We always start from a power of 0, as the right-most position and increase the power by one with each position to the left.

Binary

The binary number system (or base 2 numeral system) has two digits in its alphabet – 0 and 1. This is the natural “language” for our computers. This is because it is easy to make a stable representation of two different states, using electricity. For instance – no electricity means 0 and the presence of electricity means 1. Usually this representation is implemented by voltage presence. In most computers/microchips the standard voltage for 1 is 5 volts and the standard voltage for 0 is 0 volts.

In the past many variations existed, including negative zero (which is still in use today, but not as much as a 'zero' zero) or even 3-stated machines. However, the 3 stated machines are more difficult to build and not as stable as the binary.

To represent a number in binary you need to follow the universal rule: break down the number to a sum of exact powers of 2.

$$100_{10} = ?(\text{bin})$$

$$100 = 64 + 32 + 4$$

$$64 = 1 * 2^6$$

$$32 = 1 * 2^5$$

$$4 = 1 * 2^2$$

$$100_{10} = 1 * 2^6 + 1 * 2^5 + 0 * 2^4 + 0 * 2^3 + 1 * 2^2 + 0 * 2^1 + 0 * 2^0$$

$$100_{10} = 1100100_2$$

We don't use binary in our code. We need to understand the binary numbers in order to work with bitwise operators.

Octal :

Today the octal numeral system has limited use. Several decades ago it was used in computing and programming. It was convenient representation for systems that had 12-24-36 bit machine words.

Why? The octal digits are from 0 to 7. They are 3 bits long. That made it convenient to represent the above mentioned machine words, that are multiple of 3.

Hexadecimal :

The hexadecimal number system uses 16 digits. It includes all decimal digits from 0 to 9 and adds the first 6 letters from the English alphabet. So all symbols in hex are: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F where A(hex) = 10(dec), B = 11, C = 12, D = 13, E = 14 and F = 15. There is no rule if the letters A..F are uppercase or lowercase.

To represent a number as a hex in the source code use the “0x”(zero x) prefix. This tells the compiler that the number is in hex. The number will be converted to its decimal representation before compilation. It doesn't matter if you use decimal or hexadecimal representation of the numbers in your code - to the computer they are the same numbers.

Number representation is done in the same manner, just like with decimal and binary. The difference, of course, is that here we use a base of 16.

$$100_{10} = ?(\text{hex})$$

$$100 = 6 * 16 + 4 = 6 * 16^1 + 4 * 16^0$$

$$100_{10} = 64_{16}$$

You should already be familiar with that. Let's make one final example:

$$500_{10} = 256 + 240 + 4 = 1 * 16^2 + 15 * 16^1 + 4 * 16^0 = 1F4_{16}$$

Now you should understand that those two lines of code are identical:

```
int number = 500;
```

```
int number = 0x1F4;
```

Conversion :

Here is a small table for fast conversion between decimal, binary and hexadecimal number systems for the first 16 numbers 0..15:

Bin	0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Oct	0	1	2	3	4	5	6	7	10	11	12	13	14	15	16	17
Dec	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Hex	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F

Q) write about the binary arithmetic operations with suitable examples?
There are four binary arithmetic operations

- 1) Binary Addition
- 2) Binary subtraction
- 3) Binary multiplication
- 4) Binary division

1) Binary Addition :-
Rules by binary additions

- $0+0=0$
- $0+1=1$
- $1+0=1$
- $1+1=0$ carrying '1'

Examples

$\begin{array}{r} 111011 \\ 10011 \\ \hline 1001110 \end{array}$	$\begin{array}{r} 1110010 \\ 1101110 \\ \hline 11100000 \end{array}$
--	--

2) Binary subtraction :-
Rules by binary subtraction

- $0-0=0$
- $0-1=1$ borrowing '1'
- $1-0=1$
- $1-1=0$

Examples

$\begin{array}{r} 100101 \\ 11011 \\ \hline 011010 \end{array}$	$\begin{array}{r} 11000111 \\ 1111000 \\ \hline 01001111 \end{array}$
---	---

3) Binary multiplication :-

Binary number $\rightarrow 0, 1 \rightarrow 2$

Decimal number $\rightarrow 0 \text{ to } 9 \rightarrow 10$

Octa number $\rightarrow 0 \text{ to } 7 \rightarrow 8$

Hexa decimal number $\rightarrow 0 \text{ to } 9 \text{ and } A, B, C, D, E, F \rightarrow 16$

In hexa decimal number we considered

A-10 D-13

B-11 E-14

C-12 F-15 as follows

Examples:-

Binary to octal conversion:- Binary number

100100101110101

split into groups of 3 binary digits (bits) starting from the right

1	001	001	011	110	101	Binary
1	1	1	3	6	5	Octal

Binary to hexa decimal conversion:-

Binary number 100100101110101

split into 4 groups of bits starting from the right

1001	0010	1111	0101	Binary
9	2	F	5	Hexa decimal

Binary to Decimal:-

1) 10010₍₂₎

$1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 0 \times 2^0$

$= 16 + 2 = 18$

Decimal to Binary:-

2	84 ₍₁₀₎
2	42-0
2	21-0
2	10-1
2	5-0
2	2-1
	1-0 $\uparrow$

1010100

Decimal number into octa decimal number:-

8	249
8	31-1
8	3-7
	0-3

$= 1 \times 8^2 + 7 \times 8^1 + 2 \times 8^0$

$= 1 + 56 + 192 = 249$

$\therefore (371)_{(8)} = 249_{(10)}$

Decimal into hexa decimal number:-

16	249
16	15-9
	0-F

$\therefore (249)_{(10)} = F9_{(16)}$

Octal number into Decimal number:-

$371_{(8)} \Rightarrow 3 \times 8^2 + 7 \times 8^1 + 1 \times 8^0$

$3 \times 64 + 7 \times 8 + 1 = 249_{(10)}$

INTRODUCTION TO 'C' LANGUAGE:

C language facilitates a very efficient approach to the development and implementation of computer programs. The History of C started in 1972 at the Bell Laboratories, USA where Dennis M. Ritchie proposed this language. In 1983 the American National Standards Institute (ANSI) established committee whose goal was to produce “an unambiguous and machine independent definition of the language C “ while still retaining it's spirit .

C is the programming language most frequently associated with UNIX. Since the 1970s, the bulk of the UNIX operating system and its applications have been written in C. Because the C language does not directly rely on any specific hardware architecture, UNIX was one of the first portable operating systems. In other words, the majority of the code that makes up UNIX does not know and does not care which computer it is actually running on. Machine-specific features are isolated in a few modules within the UNIX kernel, which makes it easy for you to modify them when you are porting to a different hardware architecture.

C was first designed by Dennis Ritchie for use with UNIX on DEC PDP-11 computers. The language evolved from Martin Richard's BCPL, and one of its earlier forms was the B language, which was written by Ken Thompson for the DEC PDP-7. The first book on C was *The C Programming Language* by Brian Kernighan and Dennis Ritchie, published in 1978.

In 1983, the American National Standards Institute (ANSI) established a committee to standardize the definition of C. The resulting standard is known as *ANSI C*, and it is the recognized standard for the language, grammar, and a core set of libraries. The syntax is slightly different from the original C language, which is frequently called K&R for Kernighan and Ritchie. There is also an ISO (International Standards Organization) standard that is very similar to the ANSI standard.

It appears that there will be yet another ANSI C standard officially dated 1999 or in the early 2000 years; it is currently known as "C9X."

BASIC STRUCTURE OF C LANGUAGE

The program written in C language follows this basic structure. The sequence of sections should be as they are in the basic structure. A C program should have one or more sections but the sequence of sections is to be followed.

1. Documentation section
2. Linking section
3. Definition section
4. Global declaration section
5. Main function section
 - {
 - Declaration section
 - Executable section
 - }
6. Sub program or function section

1. DOCUMENTATION SECTION : comes first and is used to document the use of logic or reasons in your program. It can be used to write the program's objective, developer and logic details. The documentation is done in C language with `/*` and `*/`. Whatever is written between these two are called comments.

2. LINKING SECTION : This section tells the compiler to link the certain occurrences of keywords or functions in your program to the header files specified in this section.

e.g. `#include <stdio.h>`

3. DEFINITION SECTION : It is used to declare some constants and assign them some value.

e.g. `#define MAX 25`

Here `#define` is a compiler directive which tells the compiler whenever MAX is found in the program replace it with 25.

4. GLOBAL DECLARATION SECTION : Here the variables which are used through out the program (including main and other functions) are declared so as to make them global(i.e accessible to all parts of program)

e.g. `int i;` (before `main()`)

5. MAIN FUNCTION SECTION : It tells the compiler where to start the execution from

```
main()
{
    point from execution starts
}
```

main function has two sections

1. declaration section : In this the variables and their data types are declared.

2. Executable section : This has the part of program which actually performs the task we need.

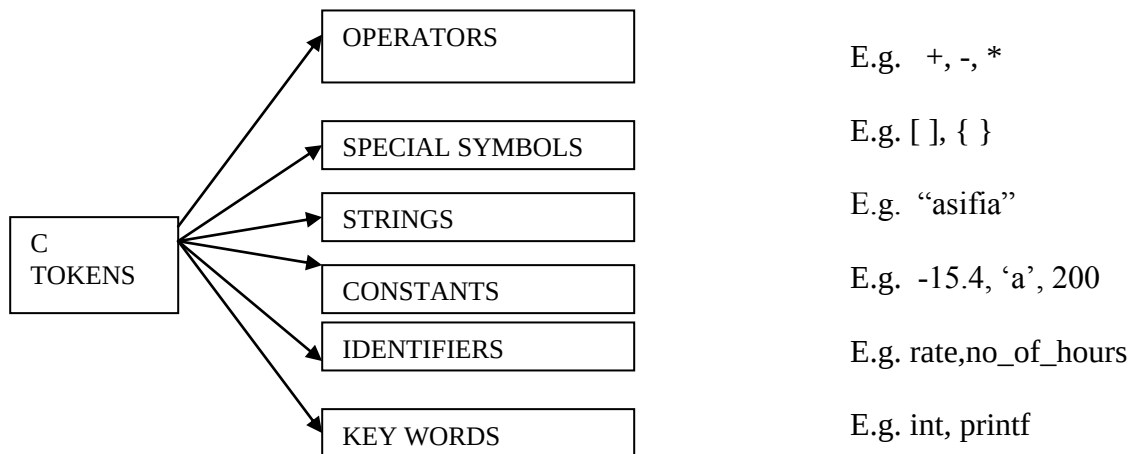
6. SUB PROGRAM OR FUNCTION SECTION : This has all the sub programs or the functions which our program needs.

SIMPLE 'C' PROGRAM:

```
/* simple program in c */
#include<stdio.h>
main()
{
    printf("welcome to c programming");
} /* End of main */
```

C-TOKENS :

Tokens are individual words and punctuations marks in English language sentence. The smallest individual units are known as C tokens.



A C program can be divided into these tokens. A C program contains minimum 3 c tokens no matter what the size of the program is.

DATA TYPES :

To represent different types of data in C program we need different data types. A data type is essential to identify the storage representation and the type of operations that can be performed on that data. C supports four different classes of data types namely

1. Basic Data types
2. Derives data types
3. User defined data types
4. Pointer data types

BASIC DATA TYPES:

All arithmetic operations such as Addition , subtraction etc are possible on basic data types.

E.g.: int a,b;

Char c;

The following table shows the Storage size and Range of basic data types:

TYPE	LENGTH	RANGE
Unsigned char	8 bits	0 to 255
Char	8 bits	-128 to 127
Short int	16 bits	-32768 to 32767
Unsigned int	32 bits	0 to 4,294,967,295
Int	32 bits	-2,147,483,648 to 2,147,483,648
Unsigned long	32 bits	0 to 4,294,967,295
Enum	16 bits	-2,147,483,648 to 2,147,483,648
Long	32 bits	-2,147,483,648 to 2,147,483,648
Float	32 bits	3.4*10E-38 to 3.4*10E38
Double	64 bits	1.7*10E-308 to 1.7*10E308
Long double	80 bits	3.4*10E-4932 to 1.1*10E4932

DERIVED DATA TYPES:

Derived datatypes are used in 'C' to store a set of data values. Arrays and Structures are examples for derived data types.

Ex: int a[10];

Char name[20];

USER DEFINED DATATYPES:

C Provides a facility called typedef for creating new data type names defined by the user. For Example ,the declaration ,

typedef int Integer;

makes the name Integer a synonym of int.Now the type Integer can be used in declarations ,casts,etc,like,

Integer num1,num2;

Which will be treated by the C compiler as the declaration of num1,num2as int variables.

“typedef” ia more useful with structures and pointers.

POINTER DATA TYPES: Pointer data type is necessary to store the address of a variable.

VARIABLES :

A quantity that can vary during the execution of a program is known as a variable. To identify a quantity we name the variable for example if we are calculating a sum of two numbers we will name the variable that will hold the value of sum of two numbers as 'sum'.

IDENTIFIERS : Names of the variables and other program elements such as functions, array, etc, are known as identifiers.

There are few rules that govern the way variable are named (identifiers).

1. Identifiers can be named from the combination of A-Z, a-z, 0-9, _(Underscore).
2. The first alphabet of the identifier should be either an alphabet or an underscore. digit are not allowed.
3. It should not be a keyword.

Eg: name, ptr, sum

After naming a variable we need to declare it to compiler of what data type it is .

The format of declaring a variable is

Data-type id1, id2, ..., idn;

where data type could be float, int, char or any of the data types.

id1, id2, id3 are the names of variable we use. In case of single variable no commas are required.

eg float a, b, c;
 int e, f, grand total;
 char present_or_absent;

ASSIGNING VALUES :

When we name and declare variables we need to assign value to the variable. In some cases we assign value to the variable directly like

a=10;

in our program.

In some cases we need to assign values to variable after the user has given input for that.

eg we ask user to enter any no and input it

/* write a program to show assigning of values to variables */

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
    int a;
```

```
    float b;
```

```
    printf("Enter any number\n");
```

```
    b=190.5;
```

```
    scanf("%d",&a);
```

```
    printf("user entered %d", a);
```

```
    printf("B's values is %f", b);
```

```
    getch();
```

```
}
```

CONSTANTS :

A quantity that does not vary during the execution of a program is known as a constant supports two types of constants namely Numeric constants and character constants.

NUMERIC CONSTANTS:

1. Example for an integer constant is 786,-127
2. Long constant is written with a terminal 'l' or 'L', for example 1234567899L is a Long constant.
3. Unsigned constants are written with a terminal 'u' or 'U', and the suffix 'ul' and 'UL' indicates unsigned long. for example 123456789u is a Unsigned constant and 1234567891ul is an unsigned long constant.
4. The advantage of declaring an unsigned constant is to increase the range of storage.
5. Floating point constants contain a decimal point or an exponent or both. For Eg : 123.4, 1e-2, 1.4E-4, etc. The suffixes f or F indicate a float constant while the absence of f or F indicate the double, l or L indicate long double.

CHARACTER CONSTANTS:

A character constant is written as one character with in single quotes such as 'a'. The value of a character constant is the numerical value of the character in the machines character set. certain character constants can be represented by escape sequences like '\n'. These sequences look like two characters but represent only one.

The following are the some of the examples of escape sequences:

Escape sequence	Description
\a	Alert
\b	Backspace
\f	Form feed
\n	New Line
\r	Carriage return
\t	Horizontal Tab
\v	Vertical Tab

String constants or string literal is a sequence of zero or more characters surrounded by a double quote. Example , " I am a little boy". quotes are not a part of the string.

To distinguish between a character constant and a string that contains a single character ex: 'a' is not

same as "a". 'a' is an integer used to produce the numeric value of letter a in the machine character set, while "a" is an array of characters containing one character and a '\0' as a string in C is an array of characters terminated by NULL.

There is one another kind of constant i.e Enumeration constant , it is a list of constant integer values.

Ex.: enum color { RED, Green, BLUE }

The first name in the enum has the value 0 and the next 1 and so on unless explicit values are specified.

If not all values specified , unspecified values continue the progression from the last specified value. For example

Enum months { JAN=1, FEB,MAR, ..., DEC }

Where the value of FEB is 2 and MAR is 3 and so on.

Enumerations provide a convenient way to associate constant values with names.

KEYWORDS :

There are certain words, called keywords (reserved words) that have a predefined meaning in 'C' language. These keywords are only to be used for their intended purpose and not as identifiers.

The following table shows the standard 'C' keywords

Auto	Break	Case	Char	Const	Continue
Default	Do	Double	Else	Enum	Extern
Float	For	Goto	If	Int	Long
Register	Return	Short	Signed	Sizeof	Static
Struct	Switch	Typedef	Union	Unsigned	void
Volatile	While				

OPERATORS :

An operator is a symbol that tells the compiler to perform certain mathematical or logical manipulations. They form expressions.

C operators can be classified as

1. Arithmetic operators
2. Relational operators
3. Logical operators
4. Assignment operators
5. Increment or Decrement operators
6. Conditional operator
7. Bit wise operators
8. Special operators

1. ARITHMETIC OPERATORS : All basic arithmetic operators are present in C.

operator	meaning
+	add
-	subtract
*	multiplication
/	division
%	modulo division(remainder)

An arithmetic operation involving only real operands(or integer operands) is called real arithmetic(or integer arithmetic). If a combination of arithmetic and real is called mixed mode arithmetic.

2. RELATIONAL OPERATORS : We often compare two quantities and depending on their relation take certain decisions for that comparison we use relational operators.

operator	meaning
<	is less than
>	is greater than
<=	is less than or equal to
>=	is greater than or equal to
==	is equal to
!=	is not equal to

It is the form of

ae-1 relational operator ae-2

3. LOGICAL OPERATORS : An expression of this kind which combines two or more relational expressions is termed as a logical expressions or a compound relational expression. The operators and truth values are

op-1	op-2	op-1 && op-2	op-1 op-2
non-zero	non-zero	1	1
non-zero	0	0	1
0	non-zero	0	1
0	0	0	0

op-1	!op-1
non-zero	zero
zero	non-zero

5. ASSIGNMENT OPERATORS : They are used to assign the result of an expression to a variable. The assignment operator is '='.

v op=exp
 v is variable
 op binary operator
 exp expression
 op= short hand assignment operator

short hand assignment operators

use of simple assignment operators

a=a+1
 a=a-1
 a=a%b

use of short hand assignment operators

a+=1
 a-=1
 a%=b

6. INCREMENT AND DECREMENT OPERATORS :

++ and == are called increment and decrement operators used to add or subtract.

Both are unary and as follows

++m or m++

--m or m--

The difference between ++m and m++ is

if m=5; y=++m then it is equal to m=5;m++;y=m;

if m=5; y=m++ then it is equal to m=5;y=m;m++;

7. CONDITIONAL OPERATOR : A ternary operator pair "?:" is available in C to construct conditional expressions of the form

exp1 ? exp2 : exp3;

It work as

if exp1 is true then exp2 else exp3

8. BIT WISE OPERATORS : C supports special operators known as bit wise operators for manipulation of data at bit level. They are not applied to float or double.

operator meaning

& Bitwise AND

	Bitwise OR
^	Bitwise exclusive OR
<<	left shift
>>	right shift
~	one's complement

9. SPECIAL OPERATORS : These operators which do not fit in any of the above classification are ,(comma), sizeof, Pointer operators(& and *) and member selection operators (. and ->). The comma operator is used to link related expressions together.

sizeof operator is used to know the sizeof operand.

```
/* programs to exhibit the use of operators */
```

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int sum, mul, modu;
    float sub, divi;
    int i,j;
    float l, m;
    printf("Enter two integers ");
    scanf("%d%d",&i,&j);
    printf("Enter two real numbers");
    scanf("%f%f",&l,&m);
    sum=i+j;
    mul=i*j;
    modu=i%j;
    sub=l-m;
    divi=l/m;
    printf("sum is %d", sum);
    printf("mul is %d", mul);
    printf("Remainder is %d", modu);
    printf("subtraction of float is %f", sub);
    printf("division of float is %f", divi);
    getch();
}
```

```
/* program to implement relational and logical */
```

```
#include<stdio.h>
main()
{
    int i, j, k;
    printf("Enter any three numbers ");
    scanf("%d%d%d", &i, &j, &k);
    if((i<j)&&(j<k))
```

```

        printf("k is largest");
    else if i<j || j>k
    {
        if i<j && j >k
            printf("j is largest");
        else
            printf("j is not largest of all");
    }
}

```

/* program to implement increment and decrement operators */

```

#include<stdio.h>
main()
{
    int i;
    printf("Enter a number");
    scanf("%d", &i);
    i++;
    printf("after incrementing %d ", i);
    i--;
    printf("after decrement %d", i);
}

```

/* program using ternary operator and assignment */

```

#include<stdio.h>
main()
{
    int i,j,large;
    printf("Enter two numbers ");
    scanf("%d%d",&i,&j);
    large=(i>j)?i:j;
    printf("largest of two is %d",large);
}

```

EXPRESSIONS : An expression is a sequence of operands and operators that reduces to a single value. Expression can be simple or complex. An **operator** is a syntactical token that requires an action to be taken. An **operand** is an object on which an operation is performed.

A simple expression contains only one operator. E.g: $2 + 3$ is a simple expression whose value is 5. A complex expression contains more than one operator. E.g: $2 + 3 * 2$. To evaluate a complex expression we reduce it to a series of simple expressions. In this first we will evaluate the simple expression $3 * 2$ (6) and then the expression $2 + 6$, giving a result of 8.

The order in which the operators in a complex expression are evaluated is determined by a set of priorities known as **precedence**, the higher the precedence, the earlier the expression containing the operator is evaluated. If two operators with the same precedence occur in a complex expression, another attribute of an operator, its associativity, takes control. **Associativity** is the parsing direction used to evaluate the expression. It can be either left-to-right or right-to-left. When two operators with the same

precedence occur in an expression and their associativity is left-to-right ,the left operator is evaluated first. For example ,in the expression $3*4/6$,there are two operators multiplication and division ,with the same precedence and left-to-right associativity .Therefore the multiplication is evaluated before the division .

The following table shows the precedence and associativity of operators:

Operators	Associativity
() [] -> .	left to right
! ~ ++ -- + - * (type) sizeof	right to left
* / %	left to right
+ -	left to right
<< >>	left to right
< <= > >=	left to right
== !=	left to right
&	left to right
^	left to right
	left to right
&&	left to right
	left to right
? :	right to left
= += -= *= /= %= &= ^= = <<= >>=	right to left
,	left to right

ARITHMETIC EXPRESSIONS : It is a combination of variables, constants and operators arranged according to the syntax of C language.

Some examples

$A * B - C \dots (M + N) * (X + Y)$

Evaluation of expressions : Expressions are evaluated using an assignment statement of the form

Variable = expression Eg $x = x*y + z - 3 * (z * y)$

Precedence of arithmetic expressions is used to evaluate a expression to provide un ambiguous way of solving an expression. The highest precedence operator is evaluated then next highest precedence operator until no operator is present.

The precedence or priorities are as follows

High * / %

Low + -

An **expression is evaluated** in left to right and value is assigned to variable in left side of assignment operator.

```
#include<stdio.h> /* program to demonstrate evaluation of expressions */
```

```
void main()
```

```
{
```

```
float a,b,c,x,y,z;
```

```
a=9;b=23;c=3;
```

```
x=a-b/3+c*2-1;
```

```
y=a-b/(3+c)*(2-1);
```

```

z=a-(b/(3+c)*2)-1;
printf("values of x,y,z are %d%d%d",x,y,z);
getch();
}

```

TYPE CONVERSION:

In an expression that involves two different data types ,such as multiplying an integer and a floating point number to perform these evaluations ,one of the types must be converted.

We have two types of conversions

1.Implicit Type Conversion

2.Explicit Type Conversion

IMPLICIT TYPE CONVERSION :

When the types of the two operands in a binary expression are different automatically converts one type to another .This is known as implicit type conversion .

EXPLICIT TYPE CONVERSION :

Explicit type conversion uses the unary cast operator ,to convert data from one type to another. To cast data from one type to another ,we specify the new type in parentheses before the value we want converted.

For example ,to convert an integer ,a , to a float, we code the expression like

(float) a

INPUT AND OUTPUT STATEMENTS :

The simplest of input operator is getchar to read a single character from the input device.

varname=getchar(); you need to declare varname.

The simplest of output operator is putchar to output a single character on the output device.

putchar(varname)

The getchar() is used only for one input and is not formatted. Formatted input refers to an input data that has been arranged in a particular format, for that we have scanf.

scanf("control string", arg1, arg2,...argn);

Control string specifies field format in which data is to be entered.

arg1, arg2... argn specifies address of location or variable where data is stored.

eg scanf("%d%d",&a,&b);

%d used for integers

%f floats

%l long

%c character

for formatted output you use printf.....printf("control string", arg1, arg2,...argn);

/* program to exhibit i/o */

#include<stdio.h>

void main()

{

int a,b;

float c;

printf("Enter any number");

a=getchar();

printf("the char is ");

putchar(a);

printf("Exhibiting the use of scanf");

printf("Enter three numbers");

```
scanf("%d%d%f",&a,&b,&c);
printf("%d%d%f",a,b,c);
getch();
}
```

STATEMENTS AND BLOCKS :

A statement causes an action to be performed by the program. It translates directly in to one or more executable computer instructions.

STATEMENT TYPES:

1.NULL STATEMENT : The null statement is just a semicolon (the terminator).

Eg: `//null statement`

Although they do not arise often, there are syntactical situations where we must have a statement but no action is required .In these situations we use the null statement.

2.EXPRESSION STATEMENT : An expression is turned in to a statement by placing a semicolon(;)after it.

expression; `//expression statement`

Eg: `a=2;`

3.RETURN STATEMENT :

A return statement terminates a function. All functions ,including main, must have a return statement. Where there is no return statement at the end of the function ,the system inserts one with a void return value.

`return expression; //return statement`

The return statement can return a value to the calling function. In case of main ,it returns a value to the operating system rather than to another function. A return value of zero tells the operating system that the program executed successfully.

4.COMPOUND STATEMENTS:

A compound statement is a unit of code consisting of zero or more statements .It is also known as a **block**. The compound statement allows a group of statements to become one single entity.

A compound statement consists of an opening brace ,an optional declaration and definition section ,and an optional statement section ,followed by a closing brace.

Eg:

```
{
    //Local Declarations
    int x;
    //Statements
    x=1;
}    //End Block
```

IF AND SWITCH STATEMENTS : We have a number of situations where we may have to change the order of execution of statements based on certain conditions or repeat a group of statements until certain specified conditions are met.

The if statement is a two way decision statement and is used in conjunction with an expression. It takes the following form

If(test expression)

If the test expression is true then the statement block after if is executed otherwise it is not executed

```
if (test expression)
{
    statement block;
}
```

statement-x ; only statement-x is executed

```
#include<stdio.h>          /* program for if */
void main()
{
    int a,b;
    printf("Enter two numbers");
    scanf("%d%d",&a,&b);
    if a>b
        printf(" a is greater");
    if b>a
        printf("b is greater");
    getch();
}
```

The if –else statement: If your have another set of statement to be executed if condition is false then if-else is used

```
if (test expression)
{
    statement block1;
}
else
{
    statement block2;
}
statement -x ;
```

```
#include<stdio.h>          /* program for if-else */
void main()
{
    int a,b;
    printf("Enter two numbers");
    scanf("%d%d",&a,&b);
    if a>b
        printf(" a is greater")
    else
        printf("b is greater");
    getch();
}
```

Nesting of if..else statement : If more than one if else statement

```
if(text cond1)
{
    if (test expression2
```



```
{
    statement block1;
}
else
{
    statement block 2;
}
}
```

```
else
{
    statement block2;
}
statement-x ;
```

if else ladder

```
if(condition1)
    statement1;
else if(condition2)
    statement 2;
else if(condition3)
    statement n;
else
    default statement.
statement-x;
```

The nesting of if-else depends upon the conditions with which we have to deal.

THE SWITCH STATEMENT : If for suppose we have more than one valid choices to choose from then we can use switch statement in place of if statements.

```
switch(expression)
{
    case value-1
        block-1
        break;
    case value-2
        block-2
        break;
    default:
        default block;
        break;
}
statement-x
```

In case of

```
if(cond1)
{
    statement-1
}
```

```
    }  
    if (cond2)  
    {  
        statement 2  
    }
```

```
#include<stdio.h> /* program to implement switch */  
void main()  
{  
    int marks,index;  
    char grade[10];  
  
    printf("Enter your marks");  
    scanf("%d",&marks);  
    index=marks/10;  
    switch(index)  
    {  
        case 10 :  
        case 9:  
        case 8:  
        case 7:  
        case 6: grade="first";  
                break;  
        case 5 : grade="second";  
                break;  
        case 4 : grade="third";  
                break;  
        default : grade ="fail";  
                break;  
    }  
    printf("%s",grade);  
    getch();  
}
```

LOOPING : Some times we require a set of statements to be executed repeatedly until a condition is met. We have two types of looping structures. One in which condition is tested before entering the statement block called entry control.

The other in which condition is checked at exit called exit controlled loop.

WHILE STATEMENT :

```
While(test condition)  
{  
    body of the loop  
}
```

It is an entry controlled loop. The condition is evaluated and if it is true then body of loop is executed. After execution of body the condition is once again evaluated and if it is true body is executed once again. This goes on until test condition becomes false.

```
#include<stdio.h>    /* program for while */
void main()
{
    int count,n;
    float x,y;
    printf("Enter the values of x and n");
    scanf("%f%d",&x,&n);
    y=1.0;

    count=1;
    while(count<=n)
    {
        y=y*x;
        count++;
    }
    printf("x=%f; n=%d; x to power n = %f",x,n,y);
}
```

DO WHILE STATEMENT : The while loop does not allow body to be executed if test condition is false. The do while is an exit controlled loop and its body is executed at least once.

```
do
{
    body
}while(test condition)
```

```
#include<stdio.h>    /* printing the sum of first 5 numbers */
void main()
{
    int a=0,sum=0;
    do
    {
        printf("Enter the value of a");
        scanf("%d",&a);
        sum=sum+a;
    }while(a<=5);
    printf("The sum of 5 numbers is %d",sum);
    getch();
}
```

THE FOR LOOP :

It is also an entry control loop that provides a more concise structure

```
for(initialization; test control; increment)
{
    body of loop
}
```

```
#include<stdio.h>          /* program of for loop */
int main()
{
    int i, a=10;
    for(i=1; i<=a; i++)
    {
        printf("%d \n", i);
    }
    return 0;
```

BREAK STATEMENT:

This is a simple statement. It only makes sense if it occurs in the body of a switch, do, while or for statement. When it is executed the control of flow jumps to the statement immediately following the body of the statement containing the break. Its use is widespread in switch statements, where it is more or less essential to get the control. The use of the break only justifiable when exceptional circumstances have happened and the loop has to be abandoned. It would be nice if more than one loop could be abandoned with a single break but that isn't how it works. Here is an example.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int i;

    for(i = 0; i < 10; i++)
    {
        if(i==5)
        {
            printf("\n coming out of the loop");
            break;
        }
        printf("%d\n", i);
    }
    return 0;
}
```

It reads a single character from the program's input before printing the next in a sequence of numbers. If '5' is typed, the break causes an exit from the loop. If you want to exit from more than one level of loop, the break is the wrong thing to use.

CONTINUE STATEMENT:

This statement has only a limited number of uses. The rules for its use are the same as for break, with the exception that it doesn't apply to switch statements. Executing a continue starts the next iteration of the smallest enclosing do, while or for statement immediately. The use of continue is largely restricted to the top of loops, where a decision has to be made whether or not to execute the rest of the body of the loop. In this example it ensures that division by zero (which gives undefined behaviour) doesn't happen.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int i;
    for(i = 0; i < 10; i++)
    {
        if(i == 5)
        {
            printf("\n Skipping %d from displaying",i);
            continue;
        }
        printf("%d", i);
    }
    return 0;
}
```

The continue can be used in other parts of a loop, too, where it may occasionally help to simplify the logic of the code and improve readability. continue has no special meaning to a switch statement, where break does have. Inside a switch, continue is only valid if there is a loop that encloses the switch, in which case the next iteration of the loop will be started.

There is an important difference between loops written with while and for. In a while, a continue will go immediately to the test of the controlling expression. The same thing in a for will do two things: first the update expression is evaluated, then the controlling expression is evaluated.

GOTO AND LABELS:

In C, it is used to escape from multiple nested loops, or to go to an error handling exit at the end of a function. You will need a *label* when you use a goto; this example shows both.

```
goto L1;
/* whatever you like here */
L1: /* anything else */
```

A label is an identifier followed by a colon. Labels have their own 'name space' so they can't clash with the names of variables or functions. The name space only exists for the function containing the label, so label names can be re-used in different functions. The label can be used before it is declared, too, simply by mentioning it in a goto statement.

Labels must be part of a full statement, even if it's an empty one. This usually only matters when you're trying to put a label at the end of a compound statement—like this.

The goto works in an obvious way, jumping to the labelled statements. Because the name of the label is only visible inside its own function, you can't jump from one function to another one.

It's hard to give rigid rules about the use of gotos but, as with the do, continue and the break (except in switch statements), over-use should be avoided. More than one goto every 3–5 functions is a symptom that should be viewed with deep suspicion.

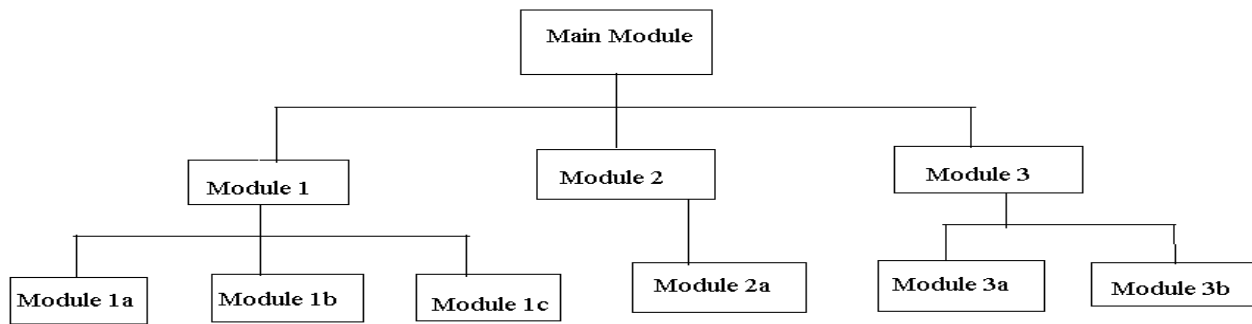
UNIT – II

DESIGNING STRUCTURED PROGRAMS:

The planning for large programs consists of first understanding the problem as a whole, second breaking it into simpler, understandable parts. We call each of these parts of a program a **module** and the process of subdividing a problem into manageable parts **top-down design**.

The principles of top-down design and structured programming dictate that a program should be divided into a main module and its related modules. Each module is in turn divided into sub-modules until the resulting modules are intrinsic; that is, until they are implicitly understood without further division.

Top-down design is usually done using a visual representation of the modules known as a structure chart. The structure chart shows the relation between each module and its sub-modules. The structure chart is read top-down, left-right. First we read Main Module. Main Module represents our entire set of code to solve the problem.



Structure Chart

Moving down and left, we then read Module 1. On the same level with Module 1 are Module 2 and Module 3. The Main Module consists of three sub-modules. At this level, however we are dealing only with Module 1. Module 1 is further subdivided into three modules, Module 1a, Module 1b, and Module 1c. To write the code for Module 1, we need to write code for its three sub-modules.

The Main Module is known as a calling module because it has sub-modules. Each of the sub-modules is known as a called module. But because Modules 1, 2, and 3 also have sub-modules, they are also calling modules; they are both called and calling modules.

Communication between modules in a structure chart is allowed only through a calling module. If Module 1 needs to send data to Module 2, the data must be passed through the calling module, Main Module. No communication can take place directly between modules that do not have a calling-called relationship.

How can Module 1a send data to Module 3b?

It first sends data to Module 1, which in turn sends it to the Main Module, which passes it to Module 3, and then on to Module 3b.

The technique used to pass data to a function is known as parameter passing. The parameters are contained in a list that is a definition of the data passed to the function by the caller. The list serves as a formal declaration of the data types and names.

FUNCTIONS :

A function is a self contained program segment that carries out some specific well defined tasks.

Advantages of functions:

1. Write your code as collections of small functions to make your program modular
2. Structured programming
3. Code easier to debug
4. Easier modification
5. Reusable in other programs

Function Definition :

```
type func_name( parameter list )
{
declarations;
statements;
}
```

FUNCTION HEADER :

```
type func_name( parameter_list )
```

Diagram illustrating the components of a function header:

- type**: type returned by the function (void if no value returned)
- func_name**: function name
- parameter_list**: list of arguments:
 - type parameter_name**: multiple arguments are separated by commas
 - void**: if no parameters

Examples:

```
int fact(int n)
void error_message(int errorcode)
double initial_value(void)
int main(void)
```

Usage:

```
a = fact(13);
error_message(2);
x=initial_value();
```

FUNCTION PROTOTYPES

If a function is not defined before it is used, it must be declared by specifying the return type and the types of the parameters.

double sqrt(double);

Tells the compiler that the function **sqrt()** takes an argument of type double and returns a double. This means, incidentally, that variables will be cast to the correct type; so **sqrt(4)** will return the correct value even though **4** is **int** not **double**. These function prototypes are placed at the top of the program, or in a separate header file, **file.h**, included as

#include "file.h"

Variable names in the argument list of a function declaration are optional:

void f(char, int);

void f(char c, int i); /*equivalent but makes code more readable */

If all functions are defined before they are used, no prototypes are needed. In this case, **main()** is the last function of the program.

TYPES OF FUNCTIONS:**1.Pre Defined Functions****2.User Defined Functions****SCOPE RULES FOR FUNCTIONS :**

Variables defined within a function (including main) are local to this function and no other function has direct access to them. The only way to pass variables to function is as parameters. The only way to pass (a single) variable back to the calling function is via the return statement

Ex:

```
int func (int n)
{
printf(“%d\n”,b);
return n;
} //b not defined locally!
```

```
int main (void)
{
int a = 2, b = 1, c;
c = func(a);
return 0;
}
```

FUNCTION CALLS :

When a function is called, expressions in the parameter list are evaluated (in no particular order!) and results are transformed to the required type. Parameters are copied to local variables for the function and function body is executed when return is encountered, the function is terminated and the result (specified in the return statement) is passed to the calling function (for example main).

Ex:

```
int fact (int n)
{
int i, product = 1;
for (i = 2; i <= n; ++i)
product *= i;
return product;
}
int main (void)
{
int i = 12;
printf(“%d”,fact(i));
return 0;
}
```

PARAMETER PASSING :

Parameter passing mechanism in ‘C’ is of two types.

1. Call by Value
2. Call by Reference.

The process of passing the actual value of variables is known as Call by Value.

The process of calling a function using pointers to pass the addresses of variables is known as Call by Reference. The function which is called by reference can change the value of the variable used in the call.

Example of Call by Value:

```
#include <stdio.h>
void swap(int,int);
main()
{
    int a,b;
    printf("Enter the Values of a and b:");

    scanf("%d%d",&a,&b);
    printf("Before Swapping \n");
    printf("a = %d \t b = %d", a,b);
    swap(a,b);
    printf("After Swapping \n");
    printf("a = %d \t b = %d", a,b);
}
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

Example of Call by Reference:

```
#include<stdio.h>
main()
{
    int a,b;
    a = 10;
    b = 20;
    swap (&a, &b);
    printf("After Swapping \n");
    printf("a = %d \t b = %d", a,b);
}
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

STORAGE CLASSES : In 'C' a variable can have any one of four Storage Classes.

1. Automatic Variables
2. External Variables
3. Static Variables
4. Register Variables

SCOPE : The Scope of variable determines over what parts of the program a variable is actually available for use.

LONGEVITY : Longevity refers to period during which a variable retains a given value during execution of a program (alive). So Longevity has a direct effect on utility of a given variable.

The variables may also be broadly categorized depending on place of their declaration as internal(local) or external(global). Internal variables are those which are declared within a particular function, while external variables are declared outside of any function.

AUTOMATIC VARIABLES :

They are declared inside a function in which they are to be utilized. They are created when function is called and destroyed automatically when the function is exited, hence the name automatic. Automatic Variables are therefore private (or local) to the function in which they are declared. Because of this property, automatic variables are also referred to as local or internal variables.

By default declaration is automatic. One important feature of automatic variables is that their value cannot be changed accidentally by what happens in some other function in the program.

```
#include<stdio.h>
void main()
{
    int m=1000;
    func2();
    printf("%d\n",m);
    getch();
}
func1()
{
    int m=10;
    printf("%d\n",m);
}
func2()
{
    int m=100;
    func1();
    printf("%d",m);
}
```

First, any variable local to main will normally live throughout the whole program, although it is active only in main.

Secondly, during recursion, nested variables are unique auto variables, a situation similar to function nested auto variables with identical names.

EXTERNAL VARIABLES :

Variables that are both alive and active throughout entire program are known as external variables. They are also known as Global Variables. In case a local and global have same name local variable will have precedence over global one in function where it is declared.

```
#include<stdio.h>
int x;
main()
{
    x=10;
    printf("%d",x);
    printf("x=%d",fun1());
    printf("x=%d",fun2());
    printf("x=%d",fun3());
}

fun1()
{
    x=x+10;
    return(x);
}
fun2()
{
    int x;
    x=1;
    return(x);
}
fun3()
{
    x=x+10;
    return(x);
}
```

An extern within a function provides the type information to just that one function.

STATIC VARIABLES :

The value of Static Variable persists until the end of program. A variable can be declared Static using Keyword Static like Internal & External Static Variables are differentiated depending whether they are declared inside or outside of auto variables, except that they remain alive throughout the remainder of program.

```
#include<stdio.h>
void main()
{
    int i;
    for (i=1;i<=3;i++)
        stat();
    getch();
}
```

```
stat()
{
    static int x=0;
    x=x+1;
    printf("x=%d\n",x);
}
```

REGISTER VARIABLES : We can tell the Compiler that a variable should be kept in one of the machines registers, instead of keeping in the memory. Since a register access is much faster than a memory access, keeping frequently accessed variables in register will lead to faster execution
Syntax: register int Count.

USER DEFINED FUNCTIONS:

Every program must have a main function to indicate where the program has to begin its execution. While it is possible to code any program utilizing only main function, it leads to a number of problems. The program may become too large and complex and as a result task of debugging, testing and maintaining becomes difficult. If a program is divided into functional parts, then each part may be independently coded and later combined into a single unit, these subprograms called “functions” are much easier to understand debug and test.

There are times when some types of operation for calculation is repeated many times at many points through out a program. For instance, we might use the factorial of a number at several points in the program. In such situations, we may repeat the program statements whenever they are needed. Another approach is to design a function that can be called and used whenever required.

The advantages of using functions are

1. It facilitates top-down modular programming.
2. The length of a source program can be reduced by using functions at appropriate places.
3. It is easy to locate and isolate a faculty function for further investigations.
4. A function may be used by many other programs.

A function is a self-contained block of code that performs a particular task. Once a function has been designed and packed it can be treated as a “black box” that takes some data from main program and returns a value. The inner details of the program are invisible to the rest of program.

The form of the C functions.

```
function-name ( argument list )
argument declaration;
{
    local variable declarations;
    executable statement 1;
    executable statement 2;
    -----
    return (expression) ;
}
```

The return statement is the mechanism for returning a value to the calling function. All functions by default returns int type data. we can force a function to return a particular type of data by using a type specifier in the header.

A function can be called by simply using the function name in the statement.

A function depending on whether arguments are present or not and whether a value is returned or not may belong to.

Four Categories of User Defined Functions:

1. Functions with no arguments and with no return values.
2. Functions with no arguments and with return values.
3. Functions with arguments and with no return values.
4. Functions with arguments and with return values.

1. Functions with no arguments and with no return values :

When a function has no arguments, then the function call will not send any data to the function definition and when the function has no return values then the function call does not receive any data from function definition. In effect, there is no data transfer between calling function and called function. (No communication)

```
#include<stdio.h>
void printhello();
void main()
{
    printhello();
    getch();
}

void printhello()
{
    printf("Hello Computer from function definition");
}
```

2. Functions with no arguments and with return values.

When a function has no arguments, then the function call will not send any data to the function definition and when the function has return values then the function call will receive some data from function definition. In effect, there is no data transfer from function call to function definition but has some data transfer from function definition to function call (One way communication)

```
#include<stdio.h>
int printhello();
void main()
{
    int a;
    a = printhello();
    printf("The number returned from function definition is %d",a);
    getch();
}

int printhello()
{
    int a=10;
    printf("Hello Computer from function definition");
    return a;
}
```

3. Functions with arguments and no return values.

When a function has arguments, then the function call will send some data to the function definition and when the function has no return values then the function call will not receive any data from function definition. In effect, there is some data transfer from function call to function definition but has no transfer from function definition to function call (One way communication)

```
#include<stdio.h>
void printhello(int);
void main()
{
    int a=10;
    printhello(a);
    getch();
}
void printhello(int x)
{
    printf("Hello Computer from function definition");
    printf("The number got from function call is %d",x);
    getch();
}
```

4. Functions with arguments and with return values.

When a function has arguments, then the function call will send some data to the function definition and when the function has return values then the function call will receive some data from function definition. In effect, there is some data transfer from function call to function definition and from function definition to function call (Two way communication).

```
#include<stdio.h>
int printhello(int);
void main()
{
    int a=10;
    a=printhello(a);

    printf("The value of a returned from function definition is %d",a);
    getch();
}

int printhello(int a)
{
    printf("Hello Computer from function definition");
    a=a+1;
    return a;
}
```

STANDARD LIBRARY FUNCTIONS AND HEADER FILES (PREDEFINED FUNCTIONS):

C functions can be classified into two categories, namely, library functions and user-defined functions. `main` is the example of user-defined functions. `printf` and `scanf` belong to the category of library functions. The main difference between these two categories is that library functions are not required to be written by us where as a user-defined function has to be developed by the user at the time of writing a program. However, a user-defined function can later become a part of the C program library.

ANSI C Standard Library Functions : The C language is accompanied by a number of library functions that perform various tasks. The ANSI committee has standardized header files which contain these functions.

Some of the Header files are:

<code><ctype.h></code>	character testing and conversion functions.
<code><math.h></code>	Mathematical functions
<code><stdio.h></code>	standard I/O library functions
<code><stdlib.h></code>	Utility functions such as string conversion routines memory allocation routines, random number generator, etc.
<code><string.h></code>	string Manipulation functions
<code><time.h></code>	Time Manipulation functions

MATH.H

The math library contains functions for performing common mathematical operations. Some of the functions are :

abs : returns the absolute value of an integer x	cos : returns the cosine of x, where x is in radians
exp: returns "e" raised to a given power	fabs: returns the absolute value of a float x
log: returns the logarithm to base e	log10: returns the logarithm to base 10
pow : returns a given number raised to another number	sqrt : returns the square root of x
sin : returns the sine of x, where x is in radians	tan : returns the tangent of x, where x is in radians
ceil : The ceiling function rounds a number with a decimal part up to the next highest integer (written mathematically as $\lceil x \rceil$)	
floor : The floor function rounds a number with a decimal part down to the next lowest integer (written mathematically as $\lfloor x \rfloor$)	

STRING.H

There are many functions for manipulating strings. Some of the more useful are:

strcat : Concatenates (i.e., adds) two strings	strcmp: Compares two strings
strlen: Calculates the length of a string (not including the null)	strcpy: Copies a string
strstr: Finds a string within another string (i.e., parts)	strtok: Divides a string into tokens

STDIO.H

Printf: Formatted printing to stdout
Fprintf: Formatted printing to a file
Getc: Get a character from a stream (e.g, stdin or a file)
putc: Write a character to a stream (e.g, stdout or a file)
fopen: Open a file

Scanf: Formatted input from stdin
Fscanf: Formatted input from a file
fgets: Get a string from a stream
fputs: Write a string to a stream
fclose: Close a file

STDLIB.H

Atof: Convert a string to a double (not a float)
Exit: Terminate a program, return an integer value
malloc: Allocate memory
number
system: Execute an external command

Atoi: Convert a string to an int
free: Release allocated memory
rand: Generate a pseudo-random

TIME.H

This library contains several functions for getting the current date and time.

Time: Get the system time

Ctime: Convert the result from time() to something meaningful

RECURSIVE FUNCTIONS :

Recursion is a repetitive process in which a function calls itself (or) A function is called recursive if it calls itself either directly or indirectly. In C, all functions can be used recursively.

Example: Fibonacci Numbers

A recursive function for Fibonacci numbers (0,1,1,2,3,5,8,13...)

```
/* Function with recursion*/
int fibonacci(int n)
{
    if (n <= 1)
        return n;
    else
        return (fibonacci(n-1) + fibonacci(n-2));
}
```

With recursion 1.4×10^9 function calls needed to find the 43rd Fibonacci number(which has the value 433494437) .If possible, it is better to write iterative functions.

```
int factorial (int n) /* iterative version */
{
    for ( ; n > 1; --n)
        product *= n;
    return product;
}
```

ARRAYS :

An array is a group of related data items that share a common name.

Ex:- Students

The complete set of students are represented using an array name students. A particular value is indicated by writing a number called index number or subscript in brackets after array name. The complete set of value is referred to as an array, the individual values are called elements.

ONE – DIMENSIONAL ARRAYS : A list of items can be given one variable index is called single subscripted variable or a one-dimensional array.

The subscript value starts from 0. If we want 5 elements the declaration will be: `int number[5];`

The elements will be `number[0], number[1], number[2], number[3], number[4]`

There will not be `number[5]`

Declaration of One - Dimensional Arrays :

Type variable – name [sizes];

Type – data type of all elements Ex: int, float etc.,

Variable – name – is an identifier

Size – is the maximum no of elements that can be stored.

Ex:- `float avg[50]`

This array is of type float. Its name is avg. and it can contains 50 elements only. The range starting from 0 – 49 elements.

Initialization of Arrays :

Initialization of elements of arrays can be done in same way as ordinary variables are done when they are declared.

Type array name[size] = {List of Value};

Ex:- `int number[3]={0,0,0};`

If the number of values in the list is less than number of elements then only that elements will be initialized. The remaining elements will be set to zero automatically.

Ex:- `float total[5]= {0.0,15.75,-10};`

The size may be omitted. In such cases, Compiler allocates enough space for all initialized elements.

`int counter[ ]= {1,1,1,1};`

/* Program showing one dimensional array */

```
#include<stdio.h>
```

```
#include<conio.h>
```

```
void main()
```

```
{
```

```
int i, a[10],total=0;
```

```
printf("Enter 10 numbers\n");
```

```
for(i=0;i<10;i++)
```

```
{
```

```
scanf("%d",&a[i]);
```

```
}
```

```
for(i=0;i<10;i++)
{
    total=total+a[i]
}
printf("total=%d",total);
getch();
}
```

TWO – DIMENSIONAL ARRAYS: To store tables we need two dimensional arrays. Each table consists of rows and columns. Two dimensional arrays are declare as

type array name [row-size][col-size];

INITIALIZING TWO DIMENSIONAL ARRAYS:

They can be initialized by following their declaration with a list of initial values enclosed in braces.

Ex:- int table[2][3] = {0,0,0,1,1,1};

Initializes the elements of first row to zero and second row to one. The initialization is done by row by row. The above statement can be written as

int table[2][3] = {{0,0,0},{1,1,1}};

When all elements are to be initialized to zero, following short-cut method may be used.

int m[3][5] = {{0},{0},{0}};

/* Write a program Showing 2-Dimensional Array (Matrix Addition)*/

```
#include<stdio.h>
#include<conio.h>
void main()
{
    int a[2][2],b[2][2],c[2][2],r1,c1,r2,c2,i,j;
    clrscr();
    printf("enter no of rows & columns of matrix-a");
    scanf("%d%d",&r1,&c1);
    printf("enter no of rows & columns of matrix-b");
    scanf("%d%d",&r2,&c2);
```

```
    if(r1==r2 && c1==c2)
    {
        printf("enter elements of matrix-a");
        for(i=0;i<r1;i++)
        {
            for(j=0;j<c1;j++)
            {
                scanf("%d",&a[i][j]);
            }
        }
        printf("enter elements of matrix-b");
```

```
for(i=0;i<r2;i++)
{
for(j=0;j<c2;j++)
{
scanf("%d",&b[i][j]);
}
}
printf("matrix addition:\n");
for(i=0;i<r1;i++)
{
for(j=0;j<c1;j++)
{
c[i][j]=a[i][j]+b[i][j];
printf("%2d",c[i][j]);
}
printf("\n");
}
}
else
printf("matrix addition is not possible");
getch();
}
```

MULTI-DIMENSIONAL ARRAYS:

C allows arrays of three or more dimensions. The exact limit is determined by Compiler.

type array-names[s1][s2][s3] - - - - [s_n];

where s_i is size of dimension.

Ex:- int Survey[3][5][2];

```
#include <stdio.h>
```

```
void printArray( const int a[][ 3 ] );
```

```
int main()
```

```
{
    int array1[ 2 ][ 3 ] = { { 1, 2, 3 }, { 4, 5, 6 } };
    int array2[ 2 ][ 3 ] = { 1, 2, 3, 4, 5 };
    int array3[ 2 ][ 3 ] = { { 1, 2 }, { 4 } };

    printf( "Values in array1 by row are:\n" );
    printArray( array1 );

    printf( "Values in array2 by row are:\n" );
    printArray( array2 );

    printf( "Values in array3 by row are:\n" );
    printArray( array3 );

    return 0;
}
```

```
void printArray( const int a[][ 3 ] )
{
    int i;
    int j;
    for ( i = 0; i <= 1; i++ )
    {
        for ( j = 0; j <= 2; j++ )
        {
            printf( "%d ", a[ i ][ j ] );
        }
        printf( "\n" );
    }
}
```

Passing arrays to a function as an argument :

Passing One-dimensional Array In Function

Single element of an array can be passed in similar manner as passing variable to a function.

Example 1:

```
#include <stdio.h>
void display(int num)
{
    printf("%d", num);
}
int main()
{
    int A[] = { 2, 3, 4 };
    display(A[2]); //Passing array element A[2] only.
    return 0;
}
```

Output: 4

Example 2:

```
void function(int array[])
{
    array[0] = 4;
    array[1] = 5;
    array[2] = 6;
}
int main()
{
    int array[] = {1,2,3};
    function(array);
    printf("%d %d %d",array[0],array[1],array[2]);
    return 0;
}
```

Output: 4 5 6

Example 3:

```
void function(int integer)
{
    integer = 2;
}
int main()
{
    int integer = 1;
    function(integer);
    printf("%d",integer);
    return 0;
}
```

Output: 1**Passing an entire one-dimensional array to a function :**

While passing arrays as arguments to the function, only the name of the array is passed (,i.e, starting address of memory area is passed as argument).

C program to pass an array containing age of person to a function. This function should find average age and display the average age in main function.

```
#include <stdio.h>
#include <conio.h>
int soa(int age[]);
int main()
{
    Int age[] = { 23, 55, 22, 3, 40, 18 },sum;
    sum = soa(age); /* Only name of array is passed as argument. */
    printf("Sum of age=%d", sum);
    return 0;
}
```

```
int soa(int age[])
{
    int i;
    int sum = 0;
    for (i = 0; i < 6; ++i)
    {
        sum += age[i];
    }
    return sum;
}
```

Output

Sum of age=161

Passing Multi-dimensional Arrays to Function :

To pass two-dimensional array to a function as an argument, starting address of memory area reserved is passed as in one dimensional array

Example: Pass two-dimensional arrays to a function

```
#include <stdio.h>
void displayNumbers(int num[2][2]);
int main()
{
    int num[2][2], i, j;
    printf("Enter 4 numbers:\n");
    for (i = 0; i < 2; ++i)
        for (j = 0; j < 2; ++j)
            scanf("%d", &num[i][j]);
    // passing multi-dimensional array to displayNumbers function
    displayNumbers(num);
    return 0;
}
void displayNumbers(int num[2][2])
{
    // Instead of the above line,
    // void displayNumbers(int num[][2]) is also valid
    int i, j;
    printf("Displaying:\n");
    for (i = 0; i < 2; ++i)
        for (j = 0; j < 2; ++j)
            printf("%d\n", num[i][j]);
}
```

Output

Enter 4 numbers: 2 3 4 5

Displaying: 2 3 4 5

APPLICATIONS OF ARRAYS :**1. Stores Elements of Same Data Type**

Array is used to store the number of elements belonging to same data type.

```
int arr[30];
```

Above array is used to store the integer numbers in an array.

```
arr[0] = 10;
```

```
arr[1] = 20;
```

```
arr[2] = 30;
```

```
arr[3] = 40;
```

```
arr[4] = 50;
```

Similarly if we declare the character array then it can hold only character. So in short character array can store character variables while floating array stores only floating numbers.

2. Array Used for Maintaining multiple variable names using single name

Suppose we need to store 5 roll numbers of students then without declaration of array we need to declare following –

```
int roll1,roll2,roll3,roll4,roll5;
```

Now in order to get roll number of first student we need to access roll1.

Guess if we need to store roll numbers of 100 students then what will be the procedure.

Maintaining all the variables and remembering all these things is very difficult.

Consider the Array :

```
int roll[5];
```

So we are using array which can store multiple values and we have to remember just single variable name.

3. Array Can be Used for Sorting Elements

We can store elements to be sorted in an array and then by using different sorting technique we can sort the elements.

Different Sorting Techniques are :

Bubble Sort

Insertion Sort

Selection Sort

Bucket Sort

4. Array Can Perform Matrix Operation

Matrix operations can be performed using the array. We can use 2-D array to store the matrix.

5. Array Can be Used in CPU Scheduling

CPU Scheduling is generally managed by Queue. Queue can be managed and implemented using the array.

Array may be allocated dynamically i.e at run time

6. Array Can be Used in Recursive Function When the function calls another function or the same function again then the current values are stores onto the stack and those values will be retrieve when control comes back. This is similar operation like stack.

UNIT – III

POINTERS :

There are Two types of Variables:

1. Normal Variables
2. Pointer Variables

Normal variables can store only values but not addresses. Ex: `int a=10;` but `a=&b` is wrong.

Pointer variables can store only addresses but not values. Ex: `int a=10,*p; p=&a;` but `p=10` is wrong.

One of the powerful features of C is ability to access the memory variables by their memory address. This can be done by using Pointers. The real power of C lies in the proper use of Pointers.

A pointer is a variable that can store an address of a variable (i.e., 112300). We say that a pointer points to a variable that is stored at that address. A pointer itself usually occupies 4 bytes of memory (then it can address cells from 0 to 232-1).

Advantages of Pointers :

1. A pointer enables us to access a variable that is defined outside the function.
2. Pointers are more efficient in handling the data tables.
3. Pointers reduce the length and complexity of a program.
4. They increase the execution speed.

Definition : A variable that holds a physical memory address is called a pointer variable or Pointer.

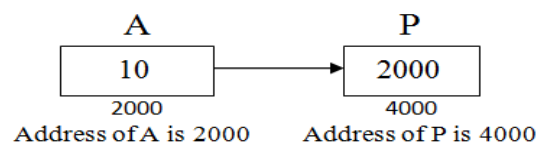
Declaration : Datatype * Variable-name;
 Eg:- `int *ad;                /* pointer to int */`
 `char *s;                /* pointer to char */`
 `float *fp;              /* pointer to float */`

A pointer is a variable that contains an address which is a location of another variable in memory.

The operator & returns the memory address of variable on which it is operated, this is called Referencing.

The * operator is called an indirection operator or dereferencing operator which is used to display the contents of the Pointer Variable.

Consider the Statement; `int A=10,*P;`
 `P=&A;`



Here '&' is called address of a variable.

'P' contains the address of a variable A.

Assume that A is stored at the memory address 2000. Then the output for the following printf statements is :

	Output
<code>Printf("The Value of A is %d",A);</code>	10
<code>Printf("The Address of A is %u",&A);</code>	2000
<code>Printf("The Address of A is %u",P);</code>	2000
<code>Printf("The Value of A is %d",*P);</code>	10
<code>Printf("The Value of A is %d",*(&A));</code>	10
<code>Printf("The Address of P is %u",&P);</code>	4000

POINTER AS A FUNCTION ARGUMENT:

Function arguments in C are strictly pass-by-value. However, we can simulate pass-by-reference by passing a pointer. This is very useful when you need to Support in/out(bi-directional) parameters (e.g. swap, find replace) Return multiple outputs (one return value isn't enough) Pass around large objects (arrays and structures).

/* Example of swapping a function can't change parameters */

```
void bad_swap(int x, int y)
{
    int temp;
    temp = x;
    x = y;
    y = temp;
}
```

/* Example of swapping - a function can't change parameters, but if a parameter is a pointer it can change the value it points to */

```
void good_swap(int *px, int *py)
{
    int temp;
    temp = *px;
    *px = *py;
    *py = temp;
}
```

```
#include <stdio.h>
#include <conio.h>
```

```
void bad_swap(int x, int y);
void good_swap(int *p1, int *p2);
```

```
void main()
{
    int a = 1, b = 999;
    printf("a = %d, b = %d\n", a, b);
    bad_swap(a, b);
    printf("a = %d, b = %d\n", a, b);
    good_swap(&a, &b);
    printf("a = %d, b = %d\n", a, b);
    getch();
}
```

POINTERS AND ARRAYS : When an array is declared, elements of array are stored in contiguous locations. The address of the first element of an array is called its base address.

Consider the array:

2000	2002	2004	2006	2008
a[0]	a[1]	a[2]	a[3]	a[4]

The name of the array is called its base address. i.e., a and &a[0] are equal.

Now both a and a[0] points to location 2000. If we declare p as an integer pointer, then we can make the pointer P to point to the array a by following assignment

P = a;

We can access every value of array a by moving P from one element to another.

i.e., P points to 0th element
 P+1 points to 1st element
 P+2 points to 2nd element
 P+3 points to 3rd element
 P +4 points to 4th element

Reading and Printing an array using Pointers :

```
main()
{
    int *a,i;
    printf("Enter five elements:");
    for (i=0;i<5;i++)
        scanf("%d",&a[i]);
    printf("The array elements are:");
    for (i=0;i<5;i++)
        printf("%d", *(a+i));
}
```

In one dimensional array, a[i] element is referred by

(a+i) is the address of ith element.

*** (a+i) is the value at the ith element.**

In two-dimensional array, a[i][j] element is referred by

***(a+i)+j is the address of ith row and jth column of the element.**

***(*(a+i)+j) is the value of ith row and jth column of the element.**

Address of 9 is represented by &A[1][2] → *(A+1)+2

Address of 3 is represented by &A[2][0] → *(A+2)+0

The value at 2nd row and 1st column is → *(* (A+2)+1)

The value at 1st row and 3rd column is → *(A+1)+3

		Column Index			
		0	1	2	3
Row Index	0	8	6	5	4
	1	2	1	9	7
	2	3	6	4	2

Two-Dimensional Array

POINTERS AND FUNCTIONS : Parameter passing mechanism in 'C' is of two types.

- 1.Call by Value
- 2.Call by Reference.

The process of passing the actual value of variables is known as Call by Value. The process of calling a function using pointers to pass the addresses of variables is known as Call by Reference. The function which is called by reference can change the value of the variable used in the call.

1. Example of Call by Value:

```
#include <stdio.h>
void swap(int,int);
void main()
{
    int a,b;
    printf("Enter the Values of a and b:");
    scanf("%d%d",&a,&b);
    printf("Before Swapping \n");
    printf("a = %d \t b = %d", a,b);
    swap(a,b);
    printf("After Swapping \n");
    printf("a = %d \t b = %d", a,b);
    getch();
}
void swap(int a, int b)
{
    int temp;
    temp = a;
    a = b;
    b = temp;
}
```

2. Example of Call by Reference:

```
#include<stdio.h>
void main()
{
    int a,b;
    a = 10;
    b = 20;
    swap (&a, &b);
    printf("After Swapping \n");
    printf("a = %d \t b = %d", a,b);
    getch();
}
void swap(int *x, int *y)
{
    int temp;
    temp = *x;
    *x = *y;
    *y = temp;
}
```

ADDRESS ARITHMETIC : Incrementing/Decrementing a pointer variable ,adding and subtracting an integer from pointer variable are all legal and known as pointer arithmetic. Pointers are valid operands in arithmetic expressions ,assignment expressions ,and comparison expressions. However not all the operators normally used in these expressions are valid in conjunction with pointer variable.

A limited set of arithmetic operations may be performed on pointers. A pointer may be incremented(+ +) or decremented(--),an integer may be added to a pointer (+ or +=),an integer may be subtracted from a pointer(- or -=),or one pointer may be subtracted from another.

We can add and subtract integers to/from pointers – the result is a pointer to another element of this type

Ex : `int *pa; char *s;`

`s-1` →points to char before s (1 subtracted)
`pa+1`→ points to next int (4 added!)
`s+9` →points to 9th char after s (9 added)
`++pa`→ increments pa to point to next int

NULL POINTER : ‘Zero’ is a special value we can assign to a pointer which does not point to anything most frequently, a symbolic constant NULL is used. It is guaranteed, that no valid address is equal to 0.The bit pattern of the NULL pointer does not have to contain all zeros usually it does or it depends on the processor architecture. On many machines, dereferencing a NULL pointer causes a segmentation violation.

NULL ptr is not the same as an EMPTY string.

```
const char* psz1 = 0;  
const char* psz2 = "";  
assert(psz1 != psz2);
```

Always check for NULL before dereferencing a pointer.

```
if (psz1)  
/* use psz1 */  
sizeof(psz1)    // doesn't give you the number of elements in psz1. Need additional size  
variable.
```

VOID POINTER :

In C ,an additional type void *(void pointer) is defined as a proper type for generic pointer. Any pointer to an object may be converted to type void * without loss of information. If the result is converted back to the original type ,the original pointer is recovered .

Ex:

```
void main()  
{  
    void *a;  
    int n=2,*m;  
    double d=2.3,*c;  
    a=&n;
```

```

m=a;

printf("\n%d %d %d",a,*m,m);
a=&d;
c=a;
printf("\n%d %3.1f %d",a,*c,c);
getch();
}

```

In the above program a is declared as a pointer to void which is used to carry the address of an int(a=&n) and to carry the address of a double(a=&d) and the original pointers are recovered with out any loss of information.

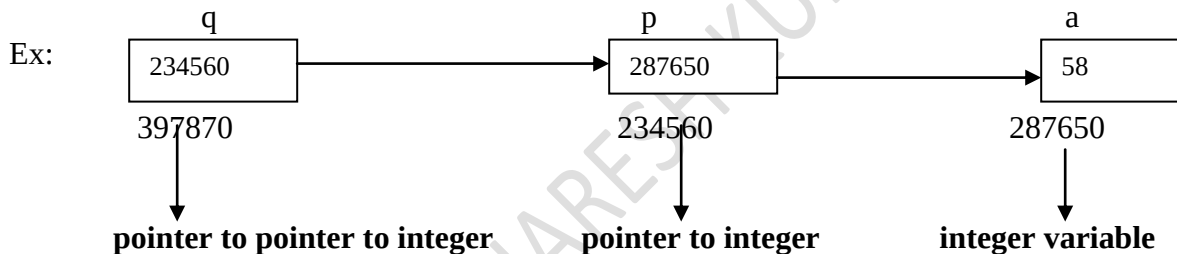
POINTERS TO POINTERS : So far ,all pointers have been pointing directly to data.It is possible and with advanced data structures often necessary to use pointers to that point to other pointers. For example,we can have a pointer pointing to a pointer to an integer.This two level indirection is seen as below:

//Local declarations

```

int a;
int* p;
int **q;

```



//statements

```

a=58;
p=&a;
q=&p;
printf("%3d",a);
printf("%3d",*p);
printf("%3d",**q);

```

There is no limit as to how many level of indirection we can use but practically we seldom use more than two. Each level of pointer indirection requires a separate indirection operator when it is dereferenced .

In the above figure to refer to 'a' using the pointer 'p', we have to dereference it as shown below.

*p

To refer to the variable 'a' using the pointer 'q', we have to dereference it twice to get to the integer 'a' because there are two levels of indirection(pointers) involved. If we dereference it only once we are referring 'p' which is a pointer to an integer .Another way to say this is that 'q' is a pointer to a pointer to an integer. The double dereference is shown below:

**q

In above example all the three references in the printf statement refer to the variable 'a'.

The first printf statement prints the value of the variable 'a' directly, second uses the pointer 'p', third uses the pointer 'q'. The result is the value 58 printed 3 times as below

58 58 58

DYNAMIC MEMORY ALLOCATION :

There are three types of allocation — **static, automatic, and dynamic.**

Static Allocation means, that the memory for your variables is allocated when the program starts. The size is fixed when the program is created. It applies to global variables, file scope variables, and variables qualified with static defined inside functions.

Automatic memory allocation occurs for (non-static) variables defined inside functions, and is usually stored on the stack (though the C standard doesn't mandate that a stack is used). You do not have to reserve extra memory using them, but on the other hand, have also limited control over the lifetime of this memory. E.g: automatic variables in a function are only there until the function finishes.

```
void func() {  
    int i; /* `i` only exists during `func` */  
}
```

Dynamic memory allocation is a bit different. You now control the exact size and the lifetime of these memory locations. If you don't free it, you'll run into memory leaks, which may cause your application to crash, since at some point of time, system cannot allocate more memory.

In C, the exact size of array is unknown until compile time, i.e., the time when a compiler compiles your code into a computer understandable language. So, sometimes the size of the array can be insufficient or more than required.

Dynamic memory allocation allows your program to obtain more memory space while running, or to release it if it's not required. In simple terms, Dynamic memory allocation allows you to manually handle memory space for your program.

Dynamic memory allocation uses predefined functions to allocate and release memory for data while the program is running. It effectively postpones the data definition, but not the declaration to run time. To use dynamic memory allocation, we use either standard data types or derived types. To access data in dynamic memory we need pointers.

MEMORY ALLOCATION FUNCTIONS:

Four memory management functions are used with dynamic memory. Three of them, malloc, calloc, and realloc, are used for memory allocation. The fourth, free, is used to return memory when it is no longer needed. All the memory management functions are found in standard library file (stdlib.h).

Function	Use of Function
malloc()	Allocates requested size of bytes and returns a pointer first byte of allocated space
calloc()	Allocates space for an array elements, initializes to zero and then returns a pointer to memory
free()	deallocate the previously allocated space
realloc()	Change the size of previously allocated space

1. BLOCK MEMORY ALLOCATION (MALLOC) :

The malloc function allocates a block of memory that contains the number of bytes specified in its parameter. It returns a void pointer to the first byte of the allocated memory. The allocated memory is not initialized.

Syntax: `ptr = (cast-type*) malloc(byte-size);`

Here, ptr is pointer of cast-type. The malloc() function returns a pointer to an area of memory with size of byte size. If the space is insufficient, allocation fails and returns NULL pointer.

`ptr = (int*) malloc(100 * sizeof(int));`

This statement will allocate either 200 or 400 according to size of int 2 or 4 bytes respectively and the pointer points to the address of first byte of memory. If it is not successful malloc returns null pointer. An attempt to allocate memory from heap when memory is insufficient is known as overflow.

2. CONTIGIOUS MEMORY ALLOCATION (CALLOC) :

Calloc is primarily used to allocate memory for arrays.

Syntax: `ptr = (cast-type*) calloc(n, element-size);`

This statement will allocate contiguous space in memory for an array of n elements. For example:

`ptr = (float*) calloc(25, sizeof(float));`

This statement allocates contiguous space in memory for an array of 25 elements each of size of float, i.e, 4 bytes.

The result is the same for both malloc and calloc. calloc returns the address of the first byte in the memory space allocated. If it is not successful calloc returns null pointer.

Example:

```
If(!(ptr=(int*)calloc(200,sizeof(int))))
    //no memory available
    exit(100);
    //memory available
...

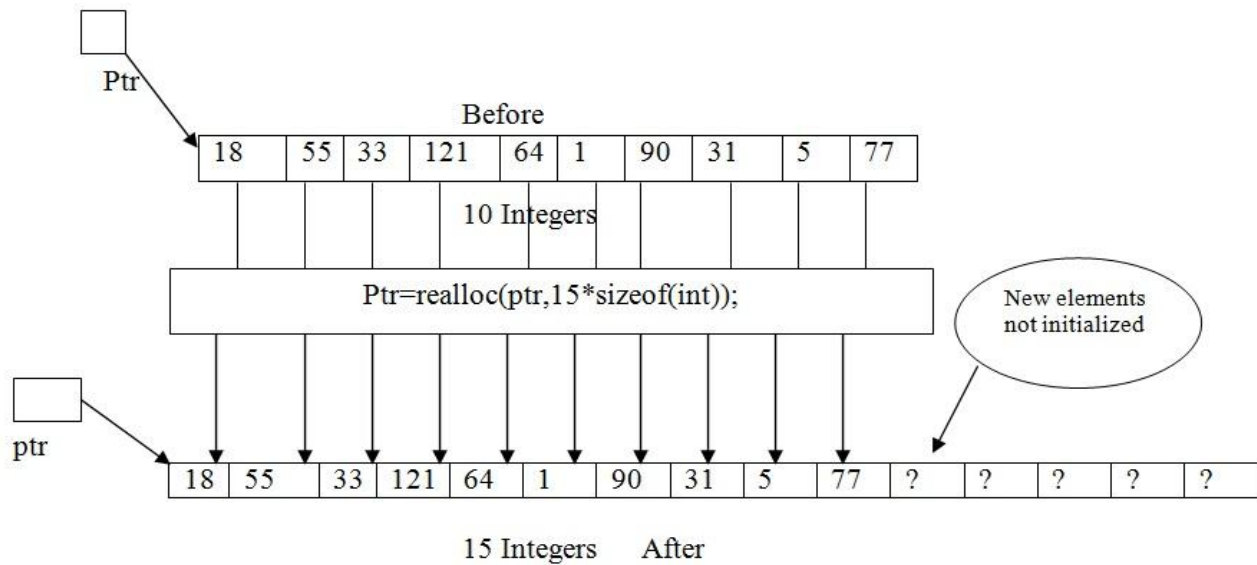
```

In this example we allocate memory for an array of 200 integers.

3. REALLOCATION OF MEMORY (REALLOC):

The realloc function can be highly inefficient and therefore should be used advisedly. When given a pointer to a previously allocated block of memory realloc changes the size of the block by deleting or extending the memory at the end of the block. If the memory can not be extended because of other allocations realloc allocates completely new block, copies the existing memory allocation to the new location, and deletes the old allocation.

Syntax: `ptr = realloc(ptr, newsize);`
Here, ptr is reallocated with size of newsize.



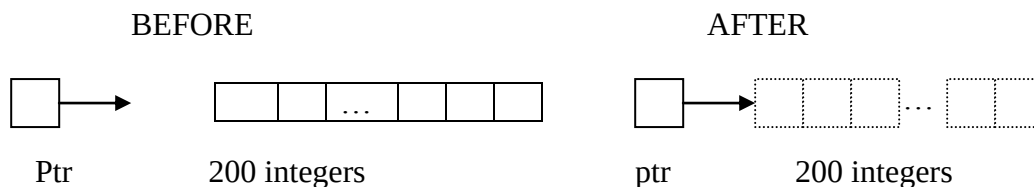
4. Releasing Memory(free): When memory locations allocated by malloc, calloc or realloc are no longer needed, they should be freed using the predefined function free. It is an error to free memory with a null pointer, a pointer to other than the first element of an allocated block, a pointer that is a different type than the pointer that allocated the memory, it is also a potential error to refer to memory after it has been released.

`void free(void *ptr);`



`Free(ptr);`

In above example it releases a single element to allocated with a malloc, back to heap.



`free(ptr);`

In the above example the 200 elements were allocated with calloc. When we free the pointer in this case, all 200 elements are return to heap. First, it is not the pointers that are being released but rather what they point to. Second , To release an array of memory that was allocated by calloc , we need only release the pointer once. It is an error to attempt to release each element individually.

Releasing memory does not change the value in a pointer. Still contains the address in the heap. It is a logic error to use the pointer after memory has been released.

Note: Memory space should be freed whenever it is not required. The operating system will release all memory when your program terminates.

Example #1: Using C malloc() and free()

Write a C program to find sum of n elements entered by user. To perform this program, allocate memory dynamically using malloc() function.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num, i, *ptr, sum = 0;
    printf("Enter number of elements: ");
    scanf("%d", &num);
    ptr = (int*) malloc(num * sizeof(int)); //memory allocated using malloc
    if(ptr == NULL)
    {
        printf("Error! memory not allocated.");
        exit(0);
    }
    printf("Enter elements of array: ");
    for(i = 0; i < num; ++i)
    {
        scanf("%d", ptr + i);
        sum += *(ptr + i);
    }
    printf("Sum = %d", sum);
    free(ptr);
    return 0;
}
```

Example #2: Using C calloc() and free()

Write a C program to find sum of n elements entered by user. To perform this program, allocate memory dynamically using calloc() function.

```
#include <stdio.h>
#include <stdlib.h>
int main()
{
    int num, i, *ptr, sum = 0;
```

```
printf("Enter number of elements: ");
scanf("%d", &num);
ptr = (int*) calloc(num, sizeof(int));
if(ptr == NULL)
{
    printf("Error! memory not allocated.");
    exit(0);
}
printf("Enter elements of array: ");
for(i = 0; i < num; ++i)
{
    scanf("%d", ptr + i);
    sum += *(ptr + i);
}

printf("Sum = %d", sum);
free(ptr);
return 0;
}
```

C realloc()

If the previously allocated memory is insufficient or more than required, you can change the previously allocated memory size using realloc().

Syntax of realloc()

```
ptr = realloc(ptr, newsize);
```

Here, ptr is reallocated with size of newsize.

Example #3: Using realloc()

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
int main()
{
    int *ptr, i, n1, n2;
    printf("Enter size of array: ");
    scanf("%d", &n1);
    ptr = (int*) malloc(n1 * sizeof(int));
    printf("Address of previously allocated memory: ");
    for(i = 0; i < n1; ++i)
        printf("%u\t", ptr + i);
    printf("\nEnter new size of array: ");
    scanf("%d", &n2);
    ptr = realloc(ptr, n2);
    for(i = 0; i < n2; ++i)
        printf("%u\t", ptr + i);
    return 0;
}
```

CHARACTER ARRAYS (STRINGS) :

A String is an array of characters. Any group of characters (except double quote sign)defined between double quotes is a constant string.

Ex: "C is a great programming language".

If we want to include double quotes.

Ex: "\"C is great \" is norm of programmers \".

Declaring and initializing strings :-

A string variable is any valid C variable name and is always declared as an array.

char string name [size];

size determines number of characters in the string name. When the compiler assigns a character string to a character array, it automatically supplies a null character ('\0') at end of String. Therefore, size should be equal to maximum number of character in String plus one.

String can be initialized when declared as

1. char city[g]= "NEW YORK";

2. char city[g]= {'N','E','W',' ','Y','O','R','K','\0'};

C also permits us to initializing a String without specifying size.

Ex:- char Strings[]= {'G','O','O','D','\0'};

READING STRINGS FROM USER:

%s format with scanf can be used for reading String.

char address[15];

scanf("%s",address);

The problem with scanf function is that it terminates its input on first white space it finds. So scanf works fine as long as there are no spaces in between the text.

Reading a line of text :

If we are required to read a line of text we use getchar(). which reads a single characters. Repeatedly to read successive single characters from input and place in character array.

/* Program to read String using scanf & getchar */

#include<stdio.h>

void main()

{

char line[80],ano_line[80],character;

int c;

c=0;

printf("Enter String using scanf to read \n");

scanf("%s", line);

printf("Using getchar enter new line\n");

do

{

character = getchar();

ano_line[c] = character;

c++;

```
    } while(character !='\n');  
    c=c-1;  
    ano_line[c]='\0';  
    getch();  
}
```

STRING INPUT/OUTPUT FUNCTIONS:

C provides two basic ways to read and write strings. First we can read and write strings with the formatted input/output functions, `scanf/fscanf` and `printf/fprintf`. Second we can use a special set of string only functions, `get string`(`gets/fgets`) and `put string`(`puts/fputs`).

Formatted string Input/Output: Declarations:

int fscanf(FILE *stream, const char *format, ...);

int scanf(const char *format, ...);

The `..scanf` functions provide a means to input formatted information from a stream

Fscanf reads formatted input from a stream

Scanf reads formatted input from stdin

These functions take input in a manner that is specified by the format argument and store each input field into the following arguments in a left to right fashion.

String Input/Output In addition to the Formatted string functions, C has two sets of string functions that read and write strings without reformatting any data. These functions convert text file lines to strings and strings to text file lines.

Line to String: gets(): Declaration: `char *gets(char *str);`

Reads a line from **stdin** and stores it into the string pointed to by *str*. It stops when either the newline character is read or when the end-of-file is reached, whichever comes first. The newline character is not copied to the string. A null character is appended to the end of the string.

On success a pointer to the string is returned. On error a null pointer is returned. If the end-of-file occurs before any characters have been read, the string remains unchanged.

fgets(): Declaration: `char *fgets(char *str, int n, FILE *stream);`

Reads a line from the specified stream and stores it into the string pointed to by *str*. It stops when either (n-1) characters are read, the newline character is read, or the end-of-file is reached, whichever comes first. The newline character is copied to the string. A null character is appended to the end of the string.

On success a pointer to the string is returned. On error a null pointer is returned. If the end-of-file occurs before any characters have been read, the string remains unchanged.

String to Line: puts:

Declaration: `int puts(const char *str);`

Writes a string to **stdout** up to but not including the null character. A newline character is appended to the output.

On success a nonnegative value is returned. On error **EOF** is returned.

fputs():

Declaration: `int fputs(const char *str, FILE *stream);`

Writes a string to the specified stream up to but not including the null character.

On success a nonnegative value is returned. On error `EOF` is returned.

ARRAY OF STRINGS:

Arrays of strings (arrays of character arrays) can be declared and handled in a similar manner to that described for 2-D arrays.

Consider the following example:

```
#include< stdio.h>
void main(void)
{
    char names[2][8] = {"Naresh", "Kumar"};
    /* Output */
    printf("names = %s, %s\n",names[0],names[1]);
    printf("names = %s\n",names);
    /* Output initials */
    printf("Initials = %c. %c.\n",names[0][0],names[1][0]);
}
```

Here we declare a 2-D character array comprising two "rows" and 8 "columns". We then initialise this array with two character strings. The output the array we need to index into each row --- using the 2-D array name on its own (strings) as a pointer cause only the first element ("row") to be produced. Note that we can still index to individual elements using index pairs. The output from the above will be:

```
names = Naresh, Kumar
names = Naresh
Initials = N. K.
```

ARRAYS OF STRINGS AS POINTER ARRAYS

The declaration of an array of character pointers is an extremely useful extension to single string pointer declarations. For example:

```
char *names[2];
```

Creates a two element array of character pointers (i.e. each element is a pointer to a character). As individual pointers each can be assigned to point to a string using a normal assignment statement:

```
names[0] = "Naresh";
names[1] = "Kumar";
```

Alternatively we can initialise the names array as shown in the following programs:

```
#include< stdio.h>
void main(void)
{
    char *names[2] = {"Naresh", "Kumar"};
    /* Output */
    printf("names = %s, %s\n",names[0],names[1]);
    /* New assignments */
    names[0] = "Siri";
```

```
names[1] = "Vennela";
/* Output */
printf("names = %s, %s\n",names[0],names[1]);
}
```

In the above the declaration of names both creates an array of pointers and initialises the pointers with appropriate addresses. Once addresses have been assigned to the pointers, each pointer can be used to access its corresponding string. The output from the above is as follows:

```
names = Naresh, Kumar
names = Siri, Vennela
```

STRING HANDLING FUNCTIONS:

strcat()	Concatenates two Strings	strcmp()	Compares two Strings
strcpy()	Copies one String Over another	strlen()	Finds length of String

strcat() : This function joins two strings together.
strcat(string1,string2);

```
string1 = VERY
string2 = FOOLISH
strcat(string1,string2);
```

```
string1=VERY FOOLISH
string2 = FOOLISH
```

strcmp() function : This function compares two strings identified by arguments and has a value 0 if they are equal. If they are not, it has the numeric difference between the first non-matching characters in the Strings.

```
strcmp(string1,string2);
Ex:- strcmp(name1,name2);
      strcmp(name1,"John");
      strcmp("ROM","Ram");
```

strcpy() function : It works almost as a string assignment operators. It takes the form
strcpy(string1,string2);
string2 can be array or a constant.

strlen() function : Counts and returns the number of characters in a string.
n= strlen(string);
n→ integer variable which receives the value of length of string.

/* Illustration of string-handling */

#include<stdio.h>

#include<string.h>

void main()

{

 char s1[20],s2[20],s3[20];

 int X,L1,L2,L3;

 printf("Enter two string constants\n");

 scanf("%s %s",s1,s2);

 X=strcmp(s1,s2);

 if (X!=0)

 {

 printf("Strings are not equal\n");

 strcat(s1,s2);

 }

 else

 printf("Strings are equal \n");

 strcpy(s3,s1);

 L1=strlen(s1);

 L2=strlen(s2);

 L3=strlen(s3);

 printf("s1=%s\t length=%d chars \n",s1,L1);

 printf("s2=%s\t length=%d chars \n",s2,L2);

 printf("s3=%s\t length=%d chars \n",s3,L3);

 getch();

}

J NARESH KUMAR

UNIT – IV

STRUCTURES : A Structure is a collection of elements of dissimilar data types. Structures provide the ability to create user defined data types and also to represent real world data.

Suppose if we want to store the information about a book, we need to store its name (String), its price (float) and number of pages in it(int). We have to store the above three items as a group then we can use a structure variable which collectively store the information as a book.

Structures can be used to store the real world data like employee, student, person etc.

Declaration : The declaration of the Structure starts with a Key Word called Struct and ends with ; . The Structure elements can be any built in types.

```
struct <Structure name>
{
    Structure element 1;
    Structure element 2;
    -
    Structure element n;
};
```

Then the Structure Variables are declared as : struct < Structure name > <Var1,Var2>;

```
Eg:- struct emp
{
    int empno.
    char ename[20];
    float sal;
};
struct emp e1,e2,e3;
```

The above Structure can also be declared as :

<pre>struct emp { int empno; char ename[20]; float sal; }e1,e2,e3;</pre>	(or)	<pre>struct { int empno; char ename[20]; float sal; }e1,e2,e3;</pre>
--	------	--

Initialization :

Structure Variables can also be initialised where they are declared.

```
struct emp
{
    int empno;
    char ename[20];
    float sal;
};
struct emp e1 = { 123,"Kumar",5000.00};
```

To access the Structure elements we use the .(dot) operator.

To refer empno we should use e1.empno

To refer sal we should use e1.sal

Structure elements are stored in contiguous memory locations as shown below. The above Structure occupies totally 26 bytes.

e1.empno	e1.ename	e1.sal
123	Kumar	5000.00
2000	2002	2022

1. Program to illustrate the usage of a Structure.

```
void main()
{
    struct emp
    {
        int empno;
        char ename[20];
        float sal;
    };
    struct emp e;
    printf("Enter Employee number: \n");
    scanf("%d",&e.empno);
    printf("Enter Employee Name: \n");
    scanf("%s",&e.ename);
    printf("Enter the Salary: \n");
    scanf("%f",&e.sal);
    printf("Employee No = %d", e.empno);
    printf("\n Emp Name = %s", e.ename);
    printf("\n Salary = %f", e.sal);
    getch();
}
```

NESTING OF STRUCTURES : One Structure can be enclosed in another Structure. The Structure which is to be nested must be declared before it is used. We can nest a Structure within a Structure, which is in still another Structure and so on. Nesting of Structure can be done to any no of levels.

The elements of the nested Structure are accessed by using one more dot(.) operator.

Eg:-

```
struct date
{
    int dd, mm, yy;
};
struct student
{
    int rno;
    char sname[15];
    struct date dob;
};
struct student s;
```

Here, dob is the Nested Structure. To access this we use s.dob.dd or s.dob.mm or s.dob.yy.

/* Program to read Student Details and Calculate total and average using structures */

```
#include<stdio.h>
void main()
{
    struct stud
    {
        int rno;
        char sname[20];
        int m1,m2,m3;
    };
    struct stud s;
    int tot;
    float avg;

    printf("Enter the student roll number: \n");
    scanf("%d",&s.rno);
    printf("Enter the student name: \n");
    scanf("%s",&s.sname);
    printf("Enter the three subjects marks: \n");
    scanf("%d%d%d",&s.m1,&s.m2,&s.m3);

    tot = s.m1 + s.m2 + s.m3;
    avg = tot/3.0;
    printf("Roll Number : %d\n",s.rno);
    printf("Student Name: %s\n",s.sname);
    printf("Total Marks : %d\n",tot);
    printf("Average : %f\n",avg);
    getch();
}
```

/*Program to illustrate the use of Nested Structures */

```
#include<stdio.h>
void main()
{
    struct date
    {
        int dd, mm, yy;
    };
    struct stud
    {
        int rno;
        char sname[15];
        struct date dob;
        int tot;
    };
    struct stud s;
    printf("Enter the student roll number: \n");
    scanf("%d",&s.rno);
    printf("Enter the student name: \n");
```

```

scanf("%s",&s.sname);
printf("Enter the date of birth of student(dd/mm/yy): \n");
scanf("%d/%d/%d",&s.dob.dd,&s.dob.mm,&s.dob.yy);

printf("Roll Number : %d\n",s.rno);
printf("Student Name: %s\n",s.sname);
printf("Date of Birth: %d/%d/%d\n", s.dob.dd,s.dob.mm,s.dob.yy);
printf("Total Marks : %f\n",s.tot);
getch();
}

```

ARRAYS OF STRUCTURES : To store more number of Structures, we can use array of Structures. In array of Structures all elements of the array are stored in adjacent memory location.

/* Program to illustrate the usage of array of Structures.

```

void main()
{
    struct book
    {
        char name[20];
        float price;
        int pages;
    };
    struct book b[10];
    int i;
    for (i=0;i<10;i++)
    {
        print("\n Enter Name, Price and Pages");
        scanf("%s%f%d", b[i].name,&b[i].price,&b[i].pages);
    }
    for (i=0;i<10;i++)
        printf("\n%s%f%d", b[i].name,b[i].price,b[i].pages);
    getch();
}

```

STRUCTURES WITH FUNCTIONS :

The entire Structure can be passed to a function at once. Whenever a Structure is passed to a function the Structure should be declared outside the main(), otherwise it will not be known to all the functions in the program.

Program to illustrate the use of Structures with functions.

```

struct stud
{
    int rno;
    char name[20];
    int tot;
};

```

```

void main()
{
    struct stud s = { 111, "Ramoo",500};
    display(s);
}
display(struct stud s)
{
    printf("Roll Number: %d\n", s.rno);
    printf("Student Name: %s\n", s.name);
    printf("Total Marks: %d\n", s.tot);
    getch();
}

```

STRUCTURE POINTER : A Pointer pointing to a struct is called Structure Pointer. A Structure Pointer can be used to store the address of a Structure Variable. An arrow operator (→) is used to refer the Structure elements when a pointer is used with a Structure.

/* Program to illustrate the usage of Structures with Pointer */

```

struct emp
{
    int empno;
    char ename[20];
    float sal;
};
void main()
{
    struct emp e;
    reademp(&e);
    displayemp(&e);
    getch();
}
reademp(struct emp *e)
{
    printf("Enter Emp Details:");
    scanf("%d%s%f", &e->empno, e->ename, &e->sal);
}
displayemp(struct emp *e)
{
    printf("empno:%d\tename:%s\tSalary=%f",e->empno,e->ename,e->sal);
}

```

SELF REFERENTIAL STRUCTURES:

A Self referential Structure is one that includes within its structure atleast one member which is a pointer to the same structure type. With self referential structures we can create very useful data structures such as linked lists,trees etc.

Example:

```

struct tag
{
    char name[40];
    struct emp*next;
};

```

A structure contains two members a 40 element character array called name and a pointer to another structure of the same called next.

APPLICATIONS OF STRUCTURES : Structures can be used for a Variety of Applications like:

- a) Database Management. (To maintain data about employees in an organisation, book in a library, items in a Store etc)
- b) Changing the size of the Cursor.
- c) Clearing the contents of the Screen.
- d) Placing the Cursor at appropriate position on Screen.
- e) Drawing any graphics Shape on the Screen.
- f) Receiving a Key from the Keyboard.
- g) Checking the memory size of the Computer.
- h) Finding the list of equipment attached to the Computer.
- i) Hiding a file from the Directory.
- j) Displaying the Directory of a Disk.
- k) Sending the Output to Printer.
- l) Interacting with the Mouse.

UNIONS :

Union, similar to Structures, are collection of elements of different data types. However, the members within a union all share the same storage area within the computer's memory, whereas each member within a Structure is assigned its own unique Storage area.

Structure enables us to treat a member of different variables stored at different places in memory, a union enables us to treat the same space in memory as a number of different variables. That is, a union offers a way for a section of memory to be treated as a variable of one type on one occasion and as a different variable of a different type on another occasion.

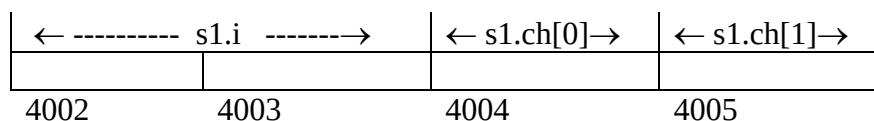
Unions are used to conserve memory. Unions are useful for applications involving multiple members, where values need not be assigned to all of the members at any given time. An attempt to access the wrong type of information will produce meaningless results.

The union elements are accessed exactly the same way in which the structure elements are accessed using dot(.) operator.

The difference between the structure and union in the memory representation is as follows.

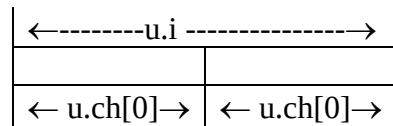
```
struct new
{
    int i;
    char ch[2];
};
struct new s1;
```

The variable s1 will occupy 4 bytes in memory and is represented as



The above datatype, if declared as union then

```
union ex
{
    int i;
    char ch[2];
}
union ex u;
```



/* Program to illustrate the use of unions */

```
void main()
{
    union example
    {
        int i;
        char ch[2];
    };
    union exaple u;

    u.i = 412;

    print("\n u.i = %d",u.i);
    print("\n u.ch[0] = %d",u.ch[0]);
    print("\n u.ch[1] = %d",u.ch[1]);

    u.ch[0] = 50; /* Assign a new value */

    print("\n\n u.i = %d",u.i);
    print("\n u.ch[0] = %d",u.ch[0]);
    print("\n u.ch[1] = %d",u.ch[1]);
    getch();
}
```

Output :

```
u.i = 512
u.ch[0] = 0
u.ch[1] =2
u.i = 562
u.ch[0] = 50
u.ch[1] =2
```

A union may be a member of a structure, and a structure may be a member of a union. And also structures and unions may be freely mixed with arrays.

/* Program to illustrate union with in a Structure */

```
main()
{
    union id
    {
        char color;
        int size;
    };
    struct
    {
        char supplier[20];
        float cost;
        union id desc;
    } pant, shirt;

    printf("%d\n", sizeof(union id));
    shirt.desc.color = 'w';
    printf("%c %d\n", shirt.desc.color, shirt.desc.size);
    shirt.desc.size = 12;
    printf("%c %d\n", shirt.desc.color, shirt.desc.size);
}
```

TYPEDEF: Typedef is used to create new datatype. The statement typedef is used while defining the new data type.

Syntax: Typedef type dataname;

Here, type is the data type and dataname is the user defined name for that type.

Typedef int hours;

Here, hours is another name for int and now we can use hours instead of int in the program as follows:

Program:

```
void main()
{
    typedef int integer;
    integer A;
    clrscr();
    printf("Enter a number\n");
    scanf("%d",&A);
    printf("Your Number is %d", A);
    getch();
}
```

output:

```
Enter a number 120
Your Number is 120
```

Explanation of program: In the above example with typedef we had declared A as an integer data type. Immediately after the typedef statement A is a variable of integer datatype which is similar to int. further the program prints the same number.

BITFIELDS: A bitfield allows structures to be subdivided into groups of bits. This process allows multiple variables to be packed into a single block of memory.

syntax: typename <variablename> : numberofbits

The type can be char, short, int, or int64 (unsigned or signed) or any of the equivalent types. If the variable name is omitted, the given number of bits will be skipped.

For example:

```
int alpha : 5;
int      : 12;
int beta  : 15;
```

Example: Suppose we want to store the following data about an employee. Each employee can:

- (a) be male or female
- (b) be single, married, divorced or widowed
- (c) have one of the eight different hobbies
- (d) can chose from any of the fifteen different schemes proposed by the company to pursue his/her hobby.

This means one bit is needed to store gender, two bits to store marital status, three for hobby and four for scheme i.e there is a need for ten bits altogether which means we can pack all this information in to a single integer since an integer is 16 bits long.

To do this using bit fields we declare the following structure:

```
struct employee
{
    unsigned gender:1;
    unsigned mat_stat:2;
    unsigned hobby:3;
    unsigned scheme:4;
};
```

The colon in the above declaration tells the compiler that we are talking about bit fields and the number after it tells how many bits to allot for the field. Once we have established a bit field, we can reference it just like any other structure element as shown in the program given below:

```
#include<stdio.h>
#define MALE 0;
#define FEMALE 1;
void main()
{
    struct employee
    {
        unsigned gender:1;
    };
    struct employee e;
    e.gender=MALE;
    printf("\nGender=%d",e.gender);
}
```

output: Gender=0

ENUMERATED DATA TYPES

In computer programming, an **enumerated type** (also called **enumeration** or **enum**) is a data type consisting of a set of named values called **elements**, **members** or **enumerators** of the type. The enumerator names are usually identifiers that behave as constants in the language. A variable that has been declared as having an enumerated type can be assigned any of the enumerators as a value.

Enumerated data type variables can only assume values which have been previously declared.

```
enum month { jan = 1, feb, mar, apr, may, jun, jul, aug, sep, oct, nov, dec };
enum month this_month;
this_month = feb;
```

In the above declaration, *month* is declared as an enumerated data type. It consists of a set of values, jan to dec. Numerically, jan is given the value 1, feb the value 2, and so on. The variable *this_month* is declared to be of the same type as month, then is assigned the value associated with feb. This_month cannot be assigned any values outside those specified in the initialization list for the declaration of month.

COMMAND LINE ARGUMENTS:

When we see the main() always it has empty parantheses. We can also pass arguments to main(). The arguments can be passed from Operating System Command Prompt. Therefore they are called as Command Line Arguments.

We have two such arguments namely argc and argv. Argc is an integer variable where as argv is array of pointers to character, i.e., an array of strings. Each string in this array will represent a argument that is passed to main. The value of argc will indicate the number of arguments passed. The Skeleton of main along with arguments will look like.

```
main(int argc, char *argv[])
{
}
}
```

In order to pass one or more arguments to the program when it is executed from the Operating System, the parameters must follow the program name on the Command line as.

Programme arg1,arg2..... argn

The individual parameters must be separated by a blank space. The program name will be stored as the first item argv, followed by each of the parameters. If there are n parameters, there will be (n+1) entries in argv. argc will automatically be assigned the value (n+1).

Eg:-

```
#include<stdio.h>
main(int argc, char *argv[])
{
    int count;
    printf("argc = %d\n",argc);
    for (count =0;count<argc;count++)
        printf("argv[%d] = %s\n", count, argv[count]);
}
```

Suppose if the above program is saved as cmdprg.c, then the command line for executing program is like

```
C:\>cmdprg one two three
Then output will be -----
argc =4
argv[0] =cmdprg
argv[1] =one
argv[2] =two
argv[3] = three.
```

/* Demonstrate the use of command-line arguments */

```
#include<stdio.h>
#include<string.h>
#include<stdlib.h>

int main(int argc,char * argv[])
{
//statements
printf("the number of arguments: %d\n",argc);
printf("the name of the program : %s\n",argv[0]);
for(int i=1; i<argc; i++)
printf("user value no. %d: %s\n ", i,argv[i]);
return 0;
} //main
```

output:

```
c:>cmdline hello
The number of arguments :2
The name of the program:cmdline
User value number.1:hello
```

C PREPROCESSOR:

C Preprocessor is a program that processes source program before it is passed to the compiler. Preprocessor commands often known as directives. We can write C programs without knowing anything about the preprocessor or its facilities but preprocessor is such a great convenience that virtually all C programmers rely on it.

FEATURES OF C PREPROCESSOR :

The C program is often known as 'source code'. The preprocessor works on the source code and creates 'Expanded source code'. If the source code is stored in a file pr1.C, then the expanded source code gets stored in a file pr1.I. It is this expanded source code that is sent to the compiler for compilation.

The preprocessor offers several features called preprocessor directives. Each of these preprocessor directives begins with a # symbol. The directives can be placed anywhere in a program but are most often placed at the beginning of a program.

PREPROCESSOR DIRECTIVES:**Preprocessor directives are:**

1. Macro expansion
2. File inclusion
3. Conditional compilation
4. Miscellaneous directives

MACRO EXPANSION:**EXAMPLE:**

```
#include<stdio.h>
#define UPPER 25
void main()
{
    int I;
    for(i=1;i<=UPPER;i++)
    printf("\n%d",i);
}
```

In this program instead of writing 25 in the for loop we are writing it in the form of UPPER, which has already been defined before main() through the statement, #define UPPER 25

This statement is called Macro definition or more commonly just a Macro.

During preprocessing, the preprocessor replaces every occurrence of UPPER in the program with 25.

MACROS WITH ARGUMENTS:

The Macros that we have used so far are called simple macros. Macros can have arguments, just as functions can.

Example:

```
#include<stdio.h>
#define AREA(x) (3.14*x*x)
void main()
{
    float r1=6.25,r2=2.5,a;
    a=AREA(r1);
    printf("\nArea of circle=%f",a);
    a=AREA(r2);
    printf("\nArea of circle=%f",a);
}
```

OUTPUT:

```
Area of circle=122.656250
Area of circle=19.625000
```

In this program, whenever the preprocessor finds the phrase AREA(x) it expands it into the statement (3.14 * x * x).The x in the macro template AREA(x) is an argument that matches the x in the macro expansion (3.14 * x * x). The statement AREA(r1) in the program causes the variable r1 to be substituted for x. Thus the statement AREA(r1) is equivalent to: (3.14*r1*r1)

After the above source code has passed through the preprocessor, the compiler gets to work on will be this:

```
#include<stdio.h>
void main()
{
    float r1=6.25,r2=2.5,a;
```

```

a=3.14*r1*r1;
printf("Area of circle=%f\n", a);
a=3.14*r2*r2;
printf("Area of circle=%f", a);
getch();
}

```

SOME IMPORTANT POINTS TO REMEMBER WHILE WRITING MACROS WITH ARGUMENTS:

- (A) Be careful not to leave a blank between the macro template and its argument while defining the macro. For example there should be no blank between AREA and (x) in the definition,

```
#define AREA(x) (3.14*x*x)
```

If we write AREA(x) instead of AREA(x), the (x) would become a part of macro expansion, which we certainly don't want. What would happen is, the template would be expanded to

```

(r1) (3.14*r1*r1)
which won't run.

```

- B) The entire macro expansion should be enclosed within parentheses.

Ex:

```

#include<stdio.h>
#define SQUARE(n) n*n
void main()
{
    int j;
    j=64/SQUARE(4);
    printf("j=%d", j);
}

```

The output of the above program will be: J=64

where as, we expected was j=4;

In this case macro was expanded into

J=64/4*4;

Which yields 64.

- C) Macros can be split into multiple lines, with a '\'(back slash) present at the end of each line.

Ex:

```

#include<stdio.h>
#define HLINE for(i=0;i<79;i++)\printf("%c",196);
#define VLINE { \ gotoxy(X,Y);\ printf("%c",179);\ }
void main()
{
    int i,y;
    clrscr();
    gotoxy(1,12);/*position cursor in row x and column y*/
    HLINE
    For(y=1;y<25;y++)
    VLINE(39,y);
}

```

This program draws a vertical and horizontal line in the center of the screen.

D) If for any reason, you are unable to debug a macro, then you should view the expanded code of the program to see how the macros are getting expanded. If your source code is present in the file pr1.c then the expanded source code would be stored in pr1.I. You need to generate this file at the command prompt by saying:

```
cpp pr1.c
```

Here CPP stands for C PreProcessor. It generates the expanded source code and stores it in a file called pr1.I. You can now open this file and see the expanded source code.

MACROS VERSUS FUNCTION:

Macro calls are like function calls, they are not really the same things. The difference between these two are:

1. In a macro call, the preprocessor replaces the macro template with its macro expansion. In function call the control is passed to a function along with certain arguments, some calculations are performed in the function and a useful value is returned back from the function.
2. Macros make the program run faster but increase the size of the program, whereas functions make the program smaller and compact.
3. If we use a macro hundred times in a program, the macro expansion goes into source code at hundred different places, thus increasing the program size. On the other hand, if a function is used, then even if it is called from hundred different places in the program, it would take the same amount of space in the program.
4. Passing parameters to a function and getting back the returned value does take time and would therefore slow down the program.
5. The macro is simple and it avoids the overhead associated with function calls. If we have a fairly large macro and it is used fairly often, we ought to replace it with a function.
6. If memory space in the machine in which the program is being executed is less, then it may make sense to use functions instead of macros. By doing so, the program may run slower, but will need less memory space.

FILE INCLUSION:

The first and most common job of a preprocessor is File inclusion, the copying of one or more files in two programs. The files are usually header files that contain function and data declarations for the program, but they can contain any valid 'C' statement.

The preprocessor command is #include and it has two different formats.

The first format is used to direct the preprocessor to include header files from the system library. In this format, the name of the header file is enclosed in pointed brackets.

The second format makes the preprocessor look for the header files in the user defined directory. In this format, the name of the file pathname is enclosed in double quotes.

Example:

```
#include <filename.h>
#include "filepath.h"
```

CONDITIONAL COMPILATION:

Conditional compilation allows us to control the compilation process by including or excluding statements.

TWO WAY COMMANDS:

The two way command tells the preprocessor to select between two choices this is similar to selection.

Syntax:

```
#if expression      //code to be included if expression is true
#else               //code to be included if expression is false
#endif
```

The expression is a constant value that evaluates to zero or non zero. If the value is nonzero, it is interpreted as true and the code after #if is included. If it is zero, it is interpreted as false and the code after #else is executed.

The #if part (or) the #else part can be empty.

Example:

```
#if expression      //code to be included if expression is true
#else               //code to be included if expression is false
#endif
```

Two abbreviations:

The C language provides two additional macro commands they are

```
#if defined name    _____> # ifdef name
#if !defined name   _____> #ifndef name
```

MULTI-WAY COMMANDS:

Conditional commands can also be multi-way by selecting one of the choices among several. In the preprocessor, we use the #elif command.

Syntax:

```
#if expression1     //code to be included if expression1 is true
#elif expression2    //code to be included if expression2 is true
#else                //code to be included if both expressions are false
#endif
```

MISCELLANEOUS DIRECTIVES: There are two or more preprocessor directives available, though they are not very commonly used. They are

- A) #undef
- B) #pragma

#undef directive: In order to undefine a macro that has been earlier #defined, we use #undef directive.

Syntax: #undef macro template
example: #undef Pentium

The above example cause the definition of Pentium to be removed from the system. All subsequent #ifdef Pentium statements would evaluate to false.

#pragma directive: This directive is another special purpose directive that you can use to turn on or off certain features. pragmas vary from one compiler to another.

Example: #pragma tokens

This command causes the compiler to perform implementation defined actions.

UNIT – V

FILES:

A file is an external collection of related data treated as a unit. The primary purpose of a file is to keep a record of data. Since the contents of primary memory are lost when the computer is shut down, we need files to store our data in a more permanent form.

Files are used to store the information permanently and to access information whenever necessary. Files are stored in the Secondary Storage Devices like Floppy Disks, Hard Disks etc. Unlike other programming languages, C does not distinguish between sequential and direct access (random access) files.

TYPES OF FILES

There are 2 kinds of files in which data can be stored in 2 ways either in characters coded in their ASCII character set or in binary format.

They are

1. Text Files.
2. Binary Files

1. TEXT FILES

A Text file contains only the text information like alphabets, digits and special symbols. The ASCII codes of these characters are stored in these files. It uses only 7 bits allowing 8 bit to be zero.

Text files are classified in to two types. These are:

A. Formatted Text files

B. Unformatted Text files

A. Formatted Text files : Formatted Text files contain variable length records must be accessed sequentially, processing all records from the start of file to access a particular record.

For formatted input and output to files we use fprintf and fscanf functions. These functions are the same as printf and scanf except that the output is directed to the stream accessed by the file pointer, fptr.

fprintf : Fprintf is used to write a record to the file.

Syntax: fprintf(spOut, "format string", address list);

fprintf returns the number of characters written or negative if an error occurs.

fscanf : Fscanf is used to Read a record from the file.

Syntax: fscanf(spIn, "format string", address list);

fscanf returns the number of input items assigned or EOF if an error or EOF occurs before any characters are input. **Rewind(FILE* ptr)** - resets the current file position to the start of the file.

B. Unformatted Text files**Standard I / O :**

fputc()	fgetc()	Individual characters
fputs()	fgets()	Character Strings

BINARY FILES

A binary file is a file that uses all 8 bits of a byte for storing the information .It is the form which can be interpreted and understood by the computer.

The only difference between the text file and binary file is the data contain in text file can be recognized by the word processor while binary file data can't be recognized by a word processor.

Binary files contain fixed length records, They are in binary format (machine readable) not a human readable format like text files, Binary files can be accessed directly (i.e. random access).

The record structure for a binary file is created using structures, They are more efficient than text files as conversion from ASCII to binary (reading) and vice versa for writing does not have to occur , They cannot be read easily by other non-C programs. Binary files are appropriate for online transaction processing systems.

E.x: Airline reservation, Order processing, Banking systems.

fwrite : The function fwrite is used to write to a binary file.

fread : The function fread is used to read from a binary file.

Binary Files Opening Modes:

“rb” → open a file in read mode
“wb” → open a file in write mode
“ab” → open a file in append mode
“rb+” → open a pre-existing file in read and write mode
“wb+” → open a file in read and write mode
“ab+” → open a file in read and append mode

Text File :

- i) Data are human readable characters.
- ii) Each line ends with a newline character.
- iii) Ctrl+z or Ctrl+d is end of file character.
- iv) Data is read in forward direction only.
- v) Data is converted into the internal format before being stored in memory.

Binary File :

- i) Data is in the form of sequence of bytes.
- ii) There are no lines or newline character.
- iii) An EOF marker is used.
- iv) Data may be read in any direction.
- v) Data stored in file are in same format that they are stored in memory.

Binary Vs Text File in C :

From the programming angle there are three main areas where text and binary mode files are different. These are:

1. Handling of newlines
2. Representation of end of file
3. Storage of numbers

Let us explore these three differences:

1. In text mode, a newline character is converted into the carriage return-linefeed combination before being written to the disk. Likewise, the carriage return-linefeed combination on the disk is converted back into a newline when the file is read by a C program. However, if a file is opened in binary mode, as opposed to text mode, these conversions will not take place.
2. The second difference between text and binary modes is in the way the end-of-file is detected. In text mode, a special character, whose ASCII value is 26, is inserted after the last character in the file to mark the end of file. If this character is detected at any point in the file, the read function would return the EOF signal to the program. As against this, there is no such special character present in the binary mode files to mark the end of file. The binary mode files keep track of the end of file from the number of character present in the directory entry of the file.
3. The last difference is storage of numbers. In text file the text and characters are stored one character per byte, as we expect. But numbers are stored as strings of characters. Thus, 65112, even though it occupies 4 bytes in memory, when transferred to the disk using `fprintf()`, would occupy 5 bytes, one byte per character. Here if large amount of data is to be stored in a disk file, using text mode may turn out to be insufficient. The solution is to open the file in binary mode and use those functions (`fread()` and `fwrite()`) which store the numbers in binary format. It means each number would occupy same number of bytes on disks as it occupies in memory.

STREAMS:

All input and output operations in C is performed with streams. Stream is a sequence of characters organized into lines (text stream), each line consists of 0 or more characters and ends with the newline character '\n'. Streams provide communications channels between files and programs. A stream can be connected to a file by opening it and the connection is broken by closing the stream.

When program execution begins, three files and their associated streams are connected to the program automatically. They are

- the standard input stream
- the standard output stream
- the standard error stream

Standard input: It is connected to the keyboard & it enables a program to read data from the keyboard.

Standard output: It is connected to the screen & it enables a program to write data to the screen.

Standard error: It is+ connected to the screen & all error messages are output to standard error.

Operating systems often allow these streams to be redirected to other devices, Standard input, standard output and standard error are manipulated with file pointers stdin, stdout and stderr.

Predefined Streams :

NAME	MEANING
stdin	Standard input (from keyboard)
Stdout	Standard output (to monitor)
Stderr	Standard error output (to monitor)
Stdaux	Standard auxiliary (both input and output)
Stdprn	Standard printer output(to printer)

File status functions :

feof (file_pointer) -- to check if end of file has been reached.

ferror (file_pointer) -- to check the error status of the file

clearerr (file_pointer) -- to reset the error status of the file

File Management Functions :

rename ("old-filename","new-filename"); -- It renames the file with the new name

remove ("filename"); -- It removes the file specified (macro)

unlink ("filename"); -- It also removes the file name

ftell (file_pointer) -- returns the current position of file pointer in terms of bytes from the beginning.

Positioning the File Pointer or File Status Functions or Random Access To File

There is no need to read each record sequentially, if we want to access a particular record. C supports these functions for random access file processing.

1. **fseek()**
2. **ftell()**
3. **rewind()**

1. Fseek is used to position the file pointer at the appropriate record in the file before reading or writing a record. The function fseek sets the file position for a stream. Subsequent reads and writes will access data in the file beginning at the new position

SYNTAX: **int fseek (FILE *fp, long offset,int origin)**

The position is set to the offset from the origin, fseek returns non zero on error

fseek takes 3 arguments

1. fp the file pointer for the file in question
2. offset is an offset in a file we want to seek to
3. origin is the position that the file pointer is set to origin can be either
 - a) SEEK_SET - beginning of file
 - b) SEEK_CUR - current position in file
 - c) SEEK_END - end of file

It may be necessary when performing a number of different types of access to a single file to reset the file pointer in certain instances.

Ex: searching for a particular record in a file may leave the file pointer in the middle of the file. To write a new record to this file the pointer should be reset to point to the end of file.

2. ftell (file_pointer) -- returns the current position of file pointer in terms of bytes from the beginning.

3. rewind : The function rewind resets the file pointer back to the start of file.

Syntax : void rewind (FILE *fp)

rewind(fp) is equivalent to fseek(fp,0,SEEK_SET).

Steps in Working with Files:

1. Establish a Buffer Area
2. Opening a File
3. Reading from a File or Writing to a File
4. Closing a File.

1. Establish a Buffer Area:

The first step is to establish a buffer area, where the information is temporarily stored while being transferred between the computers memory and the file.

The Buffer Area is established by writing

FILE *ptrvar;

FILE is a Structure which has been defined in the header file “stdio.h” (Standard input/output header file), which is a link between the Operating System and Program. It is necessary to include the header file “stdio.h”. The FILE Structure contain information about the file being used, such as its current size, its location in memory etc. This buffer area allows information to be read from or written to file more fastly.

*ptrvar is a pointer variable, which contains address of the Structure FILE.

2. Opening a File:

A File must be opened before it can be created or processed. This association the file name with the buffer area. It also specifies how the file will be utilized i.e. read-only file, write-only file, or read/write file.

We use the library function fopen() for opening a file.

ptrvar = fopen(filename, mode)

where filename and mode are strings. Filename represent the name of the file, mode represents how the file opened i.e., read only, write only etc.

The fopen() function returns a pointer to the begins of the buffer area associated with the file. A NULL value is returned if the file cannot be created or when existing file cannot be found.

The different file opening modes area listed below.

Mode	Meaning
"r"	Open an existing file for reading only
"w"	Open a new file for writing only. If the file exists, its contents are over written.
"a"	Open an existing file for appending (i.e., for adding new information at the end of the file). A new file will be created if the file does not exists.
"r+"	Open an existing file for both reading and writing.
"w+"	Open a new file for both reading and writing. If the file exists, the contents are over written
"a+"	Open an existing file for both reading and appending. A new file will be created if the file does not exists

To open the file in Binary file mention the mode along with an extra letter 'b' as "rb", "wb", "rb+" and so on.

3. Reading /Writing form/to a File:

After opening the file we can read the characters from a file or write the characters into the file by using the two library functions fgetc() and fputc().

(i) **fgetc()** ⇒ read a character from the current position and advances the pointer position to the next character, and returns the character that is read.

Eg:- ch = fgetc(ptrvar)

(ii) **fputc()** ⇒ Writes a character into the file.

Eg:- fputc(ch,ptrvar)

fscanf() and fprintf() are two functions used for formatted reading and writing of characters, Strings, integers and floats.

fread() and fwrite(): These are two functions to read and write structures with file in binary mode.

The Syntax is as follows:

fwrite(&record, sizeof(record),no of records, fileptr);

fread(&record, sizeof(record),no of records, fileptr);

The first argument is the address and the structure.

The second argument is the size of the structure in bytes.

The third argument is the number of structures to read or write.

The last argument is pointer to the file.

4. Closing the File: Finally, the file must be closed at the end of the program. This can be done by using the function fclose().

fclose(ptrvar);

It is good programming practice to close a file explicitly, even though C compiler will automatically close the file.

/* Program to Create a File */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;

    char ch;
    clrscr();
    fp = fopen("chars.txt", 'w');
    printf("Enter the characters(enter * to stop)\n");
    while((ch=getchar())!='*')
        putc(ch,fp);
    printf("File Created");
    fclose(fp);
    getch();
}
```

/* Program to Create a read an Existing File */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char ch;
    clrscr();
    fp = fopen("chars.txt", 'r');
    printf("The contents of the file are:\n");
    while((ch=fgetc(fp))!=EOF)
        printf("%c",ch);
        //fputc(ch,stdout); To print the data to the standard output screen
    fclose(fp);
    getch();
}
```

/* Program to Copy One File to Another File */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fs,*ft;
    char ch;
    clrscr();
    fs = fopen("source.txt", 'r');
    if(fs == NULL)
    {
        puts("Cannot open source file");
        exit();
    }
}
```

```
ft = fopen("target", 'w');
if(ft == NULL)
{
    puts("Cannot open target file");
    fclose(fs);
    exit();
}
while((ch=fgetc(fs))!=EOF)
    putc(ch,ft);
printf("File Copied");

fclose(fs);
fclose(ft);
getch();
}
```

/* Program to illustrate the use of fprintf() */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char itemname[20];
    int qty;
    float rate;
    char choice = 'y';
    fp = fopen ("Item", "w");
    while (choice == 'y')
    {
        printf("Enter Item Name, Quantity and Rate \n");
        scanf("%s%d%f", itemname, &qty, &rate);
        fprintf(fp, "%s%d%f\n", itemname, age, rate);
        printf("Do you want to Continue (y/n)");
        fflush(stdin); //Clears buffer of input stream and writes buffer of output stream.
        choice = getch();
    }
    fclose(fp);
    getch();
}
```

/* Program to illustrate the use of fscanf() */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char itemname[20];
    int qty;
    float rate;
```

```

fp = fopen ("Item","r");

while (fscanf(fp,"%s%d%f",itemname,&qty,&rate)!=EOF)
{
    printf("Item Name : %s\n",itemname);
    printf("Quantity =%d\n",qty);
    printf("Rate = %f\n",rate);
}
fclose(fp);
getch();
}

```

Till now, we have seen examples of writing individual items into the file. Now we see how to write a record into a file and read a record from the file by using fprintf() and fscanf() respectively. Here is the program.

```

/* Program for writing records into a file using Structures */
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char choice = 'y';
    struct emp
    {
        char name[40];
        int age;
        float bs;
    };
    struct emp e;
    fp = fopen("emp.dat", "w");
    if(fp == NULL)
    {
        puts("Cannot open file");
        exit();
    }
    while (choice == 'y')
    {
        printf("Enter Name, age and Basic Salary");
        scanf("%s%d%f", e.name, &e.age, &e.bs);
        fprintf(fp, "%s%d%f\n", e.name, e.age, e.bs);
        printf("Do you want to add one more record (y/n)");
        fflush(stdin);
        choice =getch();
    }
    fclose(fp);
    getch();
}

```


/* Program to read records from file using Structures */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    struct emp
    {
        char name[40];
        int age;
        float bs;
    };

    struct emp e;
    fp = fopen("emp.dat", "r");
    if(fp == NULL)
    {
        puts("Cannot open file");
        exit();
    }
    while (fscanf(fp, "%s%d%f", e.name, &e.age, &e.bs) != EOF)
    {
        printf("\n Employee Name : %s\n", e.name);
        printf("\n Employee Age: %d", e.age);
        printf("\n Basic Salary = %f", e.bs);
    }
    fclose(fp);
}
```

In the above programs the Structure elements are handled individually. Since the above program is opened in text mode it would occupy more number of bytes. If the number of fields in the Structure increases, writing Structures using `fprintf()`, or reading them using `fscanf()` becomes complex. So overcome these disadvantages we open the file in binary mode and also handle the structures collectively as a single one by using `fread()` and `fwrite()` function.

/* Writing records into file in Binary Mode */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    char choice = 'y';
    struct emp
    {
        char name[20];
        int age;
        float bs;
    };
}
```

```
struct emp e;
fp = fopen("Employee.dat", "wb");
if(fp == NULL)
{
    puts("Cannot open file");
    exit();
}
while (choice == 'y')
{
    printf("Enter Name, age and Basic Salary");
    scanf("%s%d%f", e.name, &e.age, &e.bs);
    fwrite(&e, sizeof(e), 1, fp);

    printf("Do you want to add one more record (y/n)");
    fflush(stdin);
    choice = getch();
}
fclose(fp);
getch();
}
```

/* Read the records from Binary File */

```
#include<stdio.h>
#include<conio.h>
void main()
{
    FILE *fp;
    struct emp
    {
        char name[20];
        int age;
        float bs;
    };
    struct emp e;
    fp = fopen("Employee.dat", "rb");
    if(fp == NULL)
    {
        puts("Cannot open file");
        exit();
    }
    while (fread(&e, sizeof(e), 1, fp))
    {
        printf("\n Employee Name : %s\n", e.name);
        printf("\n Employee Age: %d", e.age);
        printf("\n Basic Salary = %f", e.bs);
    }
    fclose(fp);
    getch();
}
```

/* Program for copying one file to another file using command line arguments */

```
#include<stdio.h>
#include<conio.h>
void main(int argc, char *argv[])
{
    FILE *fs, *ft;
    Char ch;

    if (argc!=3)
    {
        puts("In sufficient arguments");
        exit();
    }
    fs = fopen(argv[1], "r");

    if (fs= =NULL)
    {
        puts("Cannot open source file");
        exit();
    }
    ft = fopen(argv[2], "w");

    if (ft = =NULL)
    {
        puts("Cannot open target file");
        fclose(fs);
        exit();
    }
    while ((ch=fgetc(fs))!=EOF)
        fputc(ch,ft);
    fclose(fs);
    fclose(ft);
}
```

If the above program is saved as filecopy.c and executed at command line as

```
filecopy first.txt      second.txt
argv[1] will be replaced with first.txt
argv[2] will be replaced with second.txt
```

The advantages of the above program are

- There is no need to recompile the program every time.
- Source and target files can be passed from command prompt.
- Once executable file is created we cannot alter the original source file.

ERROR HANDLING DURING I/O OPERATIONS:

It is possible that an error may occur during I/O operations on a file. Typical error situations include the following:

1. Trying to read beyond the end-of-file mark
2. Device Overflow
3. Trying to use a file that has not been opened
4. Trying to perform an operation on a file, When the file is opened for another type of operation.
5. Opening a file with an invalid filename.
6. Attempting to write to a write-protected file.

If we fail to check such a read and write errors a program may behave abnormally when an error occurs an unchecked error may result in a premature termination of the program or incorrect output.

File Error Handling Functions:

C language does not provide direct support for error handling. However few method and variable defined in error.h header file can be used to point out error using return value of the function call. In C language, a function return -1 or NULL value in case of any error and a global variable errno is set with the error code. So the return value can be used to check error while programming.

C language uses the following functions to represent error

1. PERROR() : return string pass to it along with the textual representation of current errno value.

2. FERROR() : reports error occurred while read/write operations
function reports the status of the file indicated. It also takes a file pointer as it's argument and returns a non zero integer if an error has been detected up to that point during process otherwise it returns zero.

Example:

```
If(FERROR(fp)!=0)
    printf(" an error has occurred");
```

The above statement print the error message if the reading is not successful.

3. STRERROR() : is defined in string.h library. This method returns a pointer to the string representation of the current errno value.

4. FEOF() : The function feof tests for end of file.

Syntax : int FEOF(FILE *fptr)

feof accepts a pointer to a FILE, It returns non-zero if EOF otherwise returns zero.
It is used when reading a file to check whether the EOF has been reached

These are three main areas where text and binary mode files are different. There are

```
5.          If (FEOF(fp))
            Printf("End of data");
            if(fp==NULL)
                printf("File could not be opened");
```